Legends of Sword and Wand

Requirements Documentation

Table of Contents

GitHub Repository Link:
https://github.com/Kevin-Bui1/CloudComputing-Legend-of-Sword-and-Wand.git

Version	Date	Authors	Changes
0.1	2026/02/20	Kevin, Tobi	Organized document to set up for requirements design document
0.2	2026/02/22	Kevin	Created project overview and microservice architecture
0.3	2026/02/23	Kevin, Tobi	Added sequence diagrams and class diagrams
0.4	2026/02/25	Kevin, Tobi	Added microservice interfaces and test cases.

0.5	2026/03/01	Kevin, Tobi	Added CI/CD pipeline explanation, Docker deployment section, and refined architecture description.
1.0	2026/03/05	Kevin, Tobi	Edited and reviewed Deliverable 1 documentation.
1.1	2026/03/8	Kevin, Tobi	Organized document to set up for Deliverable 2 requirements
1.3	2026/03/20	Kevin, Tobi	Implemented full client game logic, updated + edited documentation components to match.
1.4	2026/03/21	Kevin, Tobi	Integrated independent services, added to TDD.
1.5	2026/03/23	Kevin, Tobi	Added Integration documentation such as diagram + tests. Added system testing implementation + documentation.
1.6	2026/03/25	Kevin, Tobi	Reconfigured Microservices Interfaces section
1.7	2026/03/26	Kevin	Updated Docker Compose + Docker Images subsections
1.8	2026/03/30	Kevin, Tobi	Updated CI/CD Pipeline section by adding Workflow subsection.
1.9	2026/04/01	Kevin, Tobi	Added complete installation manual section
2.0	2026/04/03	Kevin, Tobi	Edited and reviewed Deliverable 2 documentation.

Introduction

This document presents the Software Design Description for Legends of Sword and Wand, outlining the system architecture, major design decisions, module decomposition, and final implementation for the completed project. Focused on establishing a modular, object-oriented architecture, this document details the integrated design patterns and system interactions that drive the application. The game itself is a dungeon-based RPG featuring secure profile creation, a dynamic turn-based battle engine, PvE campaign progression, a persistent PvP matchmaking system, and seamless database integration for storing party status, match history, and campaign progress.

Major Design Decisions

This project focuses on promoting modularity, high cohesion and low coupling by dividing the system into microservices with each service being responsible for one area of the game instead of one type of code. The profile service is in charge of anything related to user accounts, the database service handles all database related tasks and this pattern applies to the remaining services. This design allows for easy maintainability as whatever needs to be added or fixed, only the service that owns it will be worked on while the rest remain untouched. It also allows for the group member in charge of a service to test and deploy the service independently and any issues that happen in one service will not affect another. This approach follows a business-oriented decomposition principle where each service owns a complete part of the game instead of a technical approach that would group all interfaces, logic, and database code separately even if they belong to the same game feature.

The profile creation and login system part separates UI, logic, and data into a layered design. LoginScreen is a Java Swing view that displays the interface for users to register or login and sends them outcome messages while taking that input and sending it to AccountManager. Behind the interface, AccountManager handles the registration and authentication logic as well as using IUserDB to contact the database. This layer design ensures each class has one specific role and changes in a layer don’t affect the other layers.

The PvE module manages the player-versus-environment gameplay. It consists of the PvEController, which serves as the entry point for PvE mode, handling campaign initialization, continuation, and retrieval. The Campaign class maintains the state of the dungeon, including the current room, the associated Party, and progression tracking. Rooms are created dynamically via the RoomFactory, and all room interactions are abstracted

through the Room class, with specialized behavior implemented in BattleRoom and InnRoom. The Party class encapsulates hero management, inventory, and resources, providing controlled access to the heroes and gold. This modular structure ensures that the PvE system is extensible, testable, and isolates gameplay logic from UI concerns.

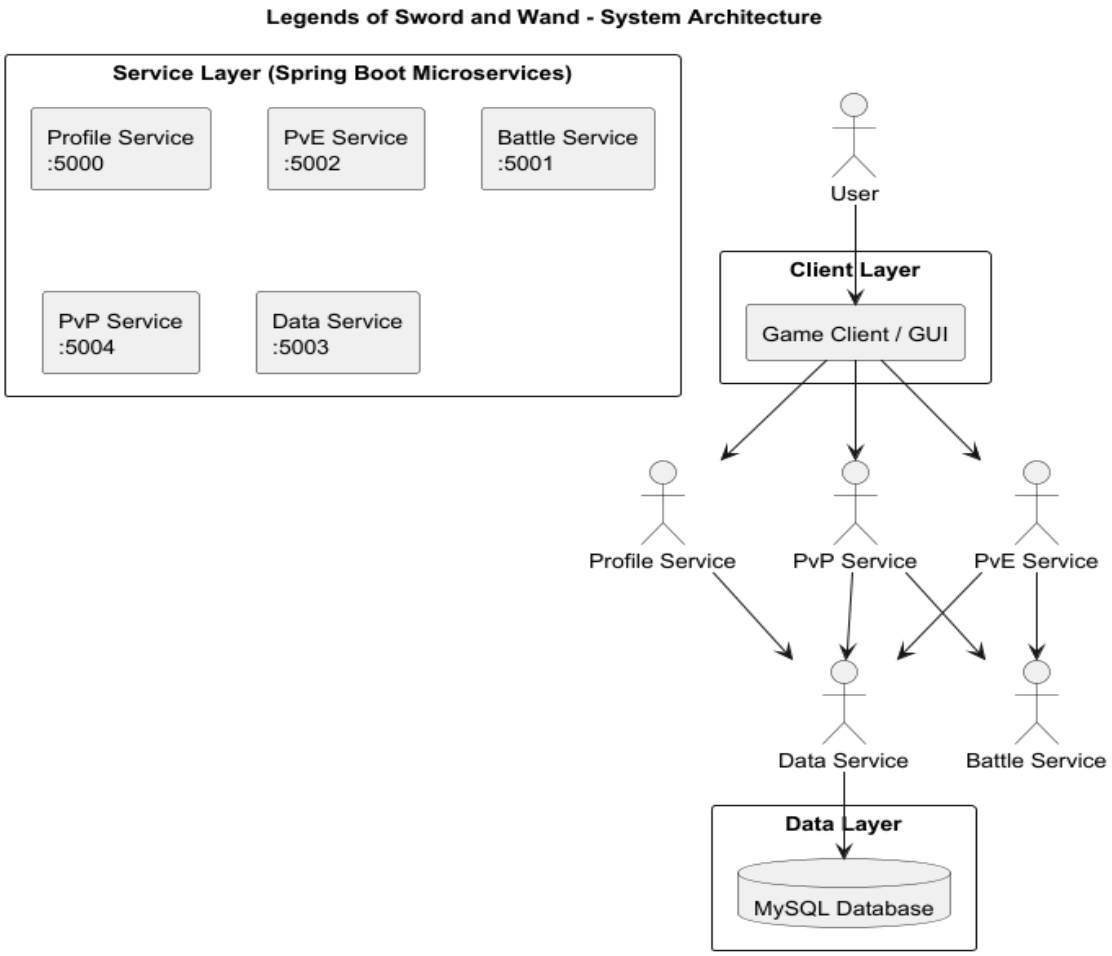
The battle system takes care of the turn-based combat between a user's party and enemies either generated or another player. Battle class contains the core logic consisting of the turn order, action, and win checking. The actions during battles are encapsulated in their own strategy class which follows the Strategy pattern allowing for future actions to be added without modifying Battle, further promoting maintainability. HeroDecorator uses decorator pattern to apply temporary stat bonuses without affecting the hero itself and BattleObserver uses the observer pattern allowing BattleGUI to listen for battle events then update the battle log so the logic and UI is not mixed.

To achieve low coupling, the database operations are separated from the rest of the system. The GameSaveDAO acts as a boundary between the app and the database by handling all SQL for users, progress, inventory, scores and PVP results. None of the other classes write SQL queries directly, instead the game logic simply passes data such as hero and party information to GameSaveDAO. This means if the database schema changes, none of the other project system parts are affected. This keeps all database implementation together, giving the persistence layer high cohesion.

Microservice Architecture

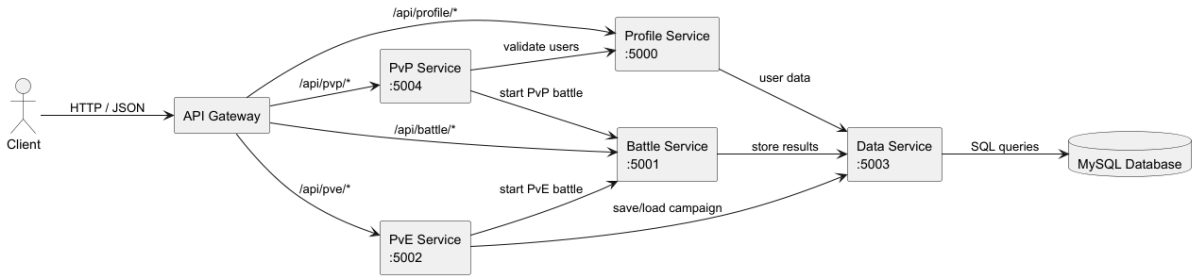
Service	Responsibility	Owner	Port
Profile	User registration and login	Kevin	5000
Battle	Battle and combat logic	Tobi	5001
PvE	Campaign progress and room generation	Tobi	5002
PvP	Player vs player invitation and match results.	Kevin	5004
Data	Database access and campaign persistence.	Kevin	5003

This system is composed of several independent microservices that communicate through REST APIs. Each service runs as a separate Docker container, allowing them to be deployed and managed independently. This separation ensures scalability and allows the individual services to be updated without causing harm to the rest of the system.



The client interface communicates with the backend through REST APIs. The profile service handles user accounts involving registration and login. PvE service takes care of user progression through campaign missions. The battle service performs the turn-based combat components and logic. The data service manages all interactions with the database.

Integration Diagram



API Gateway

In this project, an NGINX-based API Gateway is being used as the single public entry point into the system, running on port 8080. All the client requests are sent to the gateway and the gateway sends the requests to the respective service based on the URL path prefix.

The main advantage of using the API Gateway is that client interaction is simplified by exposing a single endpoint to the client. This removes the need for the client to know the location or port of each individual service. It also splits the frontend and backend services which allows the services to be modified, extended or replaced without having any effect on the client.

The gateway is also responsible for routing the requests internally, following the “smart endpoints and dumb pipes” principle, and the routing is done internally by the gateway. The API Gateway is also responsible for the scalability and maintainability of the system, as each service can be scaled individually behind the gateway without affecting the client.

URL Path Prefix	Route	Port
/profile/	Profile Service	5000
/battle/	Battle Service	5001
/pve/	PvE Service	5002
/data/	Data Service	5003
/pvp/	PvP Service	5004

Microservice Interfaces

This section describes how each microservice in this system is accessed. Each microservice provides a set of REST API endpoints that other services or applications can call, communicates using RESTful HTTP APIs and exchanges data in JSON format.

COMMON RULES
Protocol: HTTP
Data format: JSON
Request Type: application/json
Response Type: application/json

COMMON RESPONSE CODES
200 OK = request successful
201 Created = new resource created
400 Bad Request = invalid input

401 Unauthorized = invalid credentials
404 Not Found = requested data does not exist

Profile Service

Purpose: Manages user accounts including registration, login, profile retrieval, and stat updates.

Endpoint	Method	Path	Request Body	Response
Register User	POST	/api/profile/register	{"username": "alice", "password": "secret123"}	200: {"success": true, "userId": 42, "username": "alice", "scores": 0, "rankings": 0, "campaignProgress": 0} 400: {"success": false, "message": "Username already exists"}
Login	POST	/api/profile/login	{"username": "alice", "password": "secret123"}	200: {"success": true, "userId": 42, "username": "alice", "scores": 1500, "rankings": 3, "campaignProgress": 15} 401: {"success": false, "message": "Invalid credentials"}
Get Profile	GET	/api/profile/{userId}	none — userId is a path variable	200: same shape as login success 404: empty body
Update Stats	PATCH	/api/profile/{userId}/stats?newScore={n}&ranking={r}	none — both values are query params	200: updated profile object 404: empty body
Health Check	GET	/api/profile/health	none	200: {"status": "ok"}

Battle Service

Purpose: Handles stateless turn-based combat. All battle states are passed in each request so nothing is stored between calls. The battleId path variable is a caller-generated string used to identify the session.

Endpoint	Method	Path	Request Body	Response
Start Battle	POST	/api/battle/{battleId}/start	{"playerParty": [{ "name": "Aldric", "level": 3, "attack": 12, "defense": 5, "hp": 80, "maxHp": 100, "mana": 30, "maxMana": 50, "stunned": false, "alive": true, "heroClass": "WARRIOR"}], "enemyParty": [...]}	200: {"actionResult": "Battle started", "playerParty": [...], "enemyParty": [...], "battleOver": false, "winner": null}
Perform Turn	POST	/api/battle/{battleId}/action?action={ACTION}	none — action is a query param. Must be one of: ATTACK, DEFEND, CAST, WAIT	200: {"actionResult": "Aldric attacks Goblin for 9 damage!", "playerParty": [...], "enemyParty": [...], "battleOver": false, "winner": null} When battle ends: "battleOver": true, "winner": "Heroes win!" or "Enemies win!"
Health Check	GET	/api/battle/health	none	200: {"status": "ok"}

PvE Service

Purpose: Manages campaign state and room progression per user. Internally calls the Battle Service for combat rooms and the Data Service for persistence. Campaign state is held in memory per userId.

Endpoint	Method	Path	Request Body	Response
Start Campaign	POST	/api/pve/{userId}/start	[{"name": "Aldric", "heroClass": "WARRIOR", "level": 1, "attack": 5, "defense": 5, "hp": 100, "maxHp": 100, "mana": 50, "maxMana": 50}]	200: {"success": true, "message": "Campaign started!", "currentRoom": 0, "gold": 0, "cumulativeLevel": 1, "roomType": null}
Next Room	POST	/api/pve/{userId}/next-room	none	200 INN: {"success": true, "roomType": "INN",

Endpoint	Method	Path	Request Body	Response
				"currentRoom": 1, "gold": 0} 200 BATTLE: {"success": true, "roomType": "BATTLE", "currentRoom": 2, "enemies": [...], "expReward": 150, "goldReward": 225} 400: {"success": false, "message": "No active campaign"}
Get Campaign	GET	/api/pve/{userId}/campaign	none	200: CampaignResponse object 404: empty body
Restore Campaign	POST	/api/pve/{userId}/restore	{"currentRoom": 15, "gold": 750, "heroes": [...]}	200: {"success": true, "currentRoom": 15, "gold": 750, ...}
Get Score	GET	/api/pve/{userId}/score	none	200: plain integer e.g. 1750 Formula: (hero levels x 100) + (gold x 10)
End Campaign	POST	/api/pve/{userId}/end	none	200: empty body
Health Check	GET	/api/pve/health	none	200: {"status": "ok"}

Data Service

Purpose: Stores and gets game data such as user profiles, battle results, and campaign saves.

Endpoint	Method	Path	Request Body	Response
Save Campaign Data	POST	/api/data/campaign/save	{"userId": 42, "partyName": "Round Table", "currentRoom": 12, "gold": 500, "heroes": [{ "name": "Aldric", "heroClass": "WARRIOR", "level": 3, "attack": 12, "defense": 5, "hp": 80, "maxHp": 100, "mana": 30, "maxMana": 50, "experience": 120}]}	200: saved Party object with database-assigned partyId
Get Campaign Data	GET	/api/data/campaign/{userId}	none — userId is a path variable	200: {"partyId": 7, "userId": 42, "partyName": "Round Table", "currentRoom": 12, "gold": 500, "heroes": [...]} 404: empty body
Complete Campaign	PATCH	/api/data/campaign/{userId}/complete	none	200: empty body
Get Saved Parties	GET	/api/data/parties/{userId}	none	200: array of Party objects
Delete Party	DELETE	/api/data/party/{partyId}	none	200: empty body
Save Score	POST	/api/data/scores/{userId}	{"score": 1500}	200: empty body
Get Best Score	GET	/api/data/scores/{userId}/best	none	200: {"bestScore": 1500}
Get Top Scores	GET	/api/data/scores/top?limit={n}	none — limit is a query param	200: [{"username": "alice", "score": 2400}, ...]
Health Check	GET	/api/data/health	none	200: {"status": "ok"}

PvP Service

Purpose: Manages PvP invitations and records match outcomes. Battle logic is passed to the Battle Service, player and party data is loaded from the Data Service.

Endpoint	Method	Path	Request Body	Response
Send PvP Invitation	POST	/api/pvp/invite	{"fromUserId": 42, "toUsername": "bob"}	200: {"inviteId": 17, "status": "PENDING"} 400: {"error": "User not found"}
Accept Invitation	POST	/api/pvp/accept	{"inviteId": 17, "toUserId": 55}	200: {"status": "ACCEPTED"} 400: {"error": "Invite not found"}
Report Match Result	POST	/api/pvp/result	{"winnerUserId": 42, "loserUserId": 55}	200: {"status": "RECORDED", "winnerUserId": 42, "loserUserId": 55}
Health Check	GET	/api/pvp/health	none	200: {"status": "ok"}

CI/CD Pipeline

This project used GitHub Actions for continuous integration and deployment.
The pipeline performs these steps:

1. Build each microservice using Maven
2. Run automated unit tests
3. Builds and pushes Docker images for each service using repository secrets (DOCKERHUB_USERNAME and DOCKERHUB_TOKEN)
4. Starts containers using Docker Compose to ensure the system runs correctly.

This pipeline ensures changes to the code that are pushed to the repository are automatically tested.

GitHub Actions Workflow

This project contains six workflow files inside .github/workflows/. The first five files are responsible for individual service pipelines where each is triggered only when the files in that service's folder are changed. The system-integration file is the entire system integration pipeline that runs on every push to main. It builds all services together, starts the Docker stack and runs the smoke tests.

```yaml
name: Battle Service CI/CD

on:
  push:
    paths:
      - 'battle-service/**'
      - '.github/workflows/battle-service.yml'
  pull_request:
    paths:
      - 'battle-service/**'

jobs:
  test-and-build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Java 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: maven

      - name: Run unit tests
        working-directory: battle-service
        run: mvn test --no-transfer-progress

      - name: Build JAR
        working-directory: battle-service
        run: mvn package -DskipTests --no-transfer-progress

      - name: Build Docker image
        working-directory: battle-service
        run: docker build -t legends/battle-service:${{ github.sha }} .

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Push Docker image
        run: |
          docker tag legends/battle-service:${{ github.sha }} ${{ secrets.DOCKERHUB_USERNAME }}/battle-service:latest
          docker push ${{ secrets.DOCKERHUB_USERNAME }}/battle-service:latest
```

Docker Images

Each microservice in the system is packaged as a Docker container which allows the services to be deployed independently and ensures consistent environments across machines. A Dockerfile is provided for each service. Docker Compose is used to start all services together including the database.

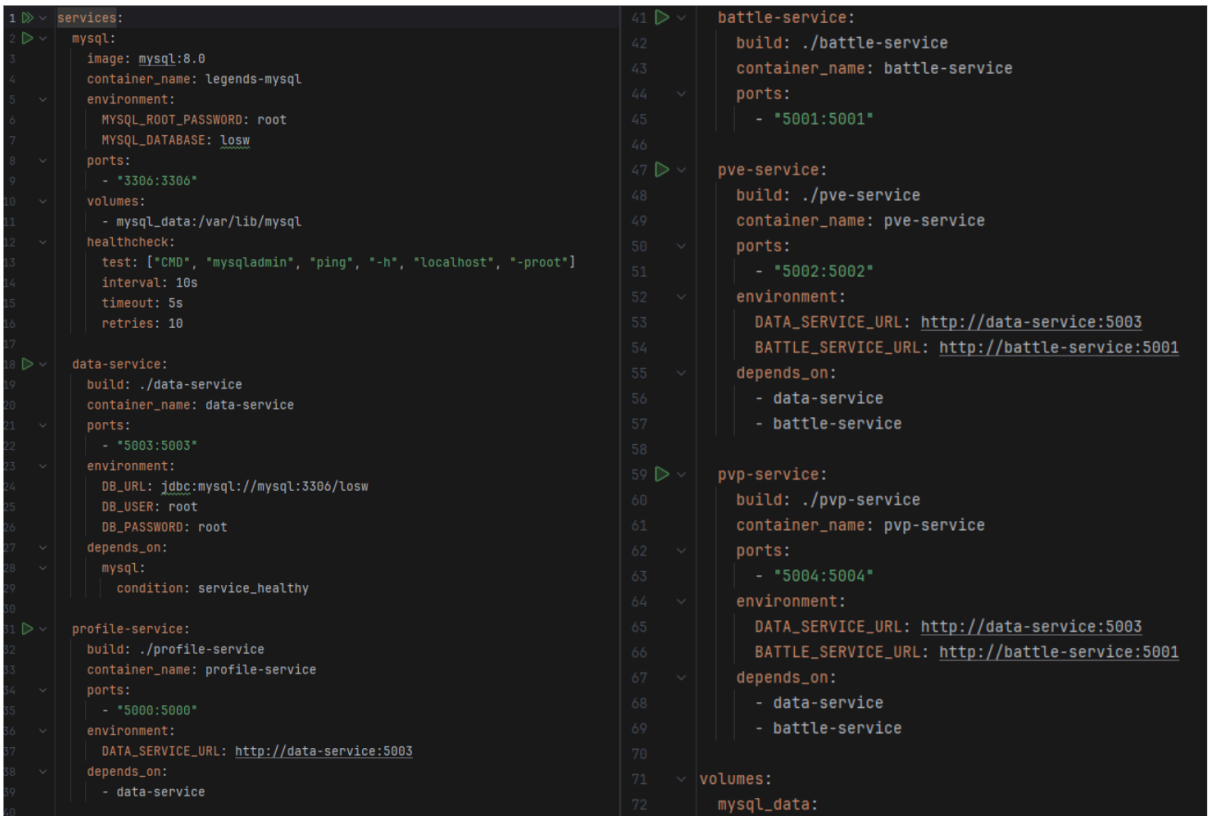
Example

```dockerfile
FROM maven:3.9.6-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -q
COPY src ./src
RUN mvn package -DskipTests -q

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
RUN addgroup -S legends && adduser -S legends -G legends
USER legends
EXPOSE 5001
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Battle-service's Dockerfile functions in two steps with the first downloading all the tools required to build the project and compiling the code into one runnable JAR file. The second step creates a new clean container that only contains Java and the JAR file, the build tools are not included which keeps the container small. A separate user account is also created so the application does not run as an administrator increasing safety. All of the services use this structure where the only difference is the port number each service listens on.

Docker Compose

Docker Compose is used to start the system entirely using one single command. The docker-compose.yml file lists all six parts of the system including the five microservices and the MySQL database. It configures how they connect to each other, what ports they use and the order in which they start.

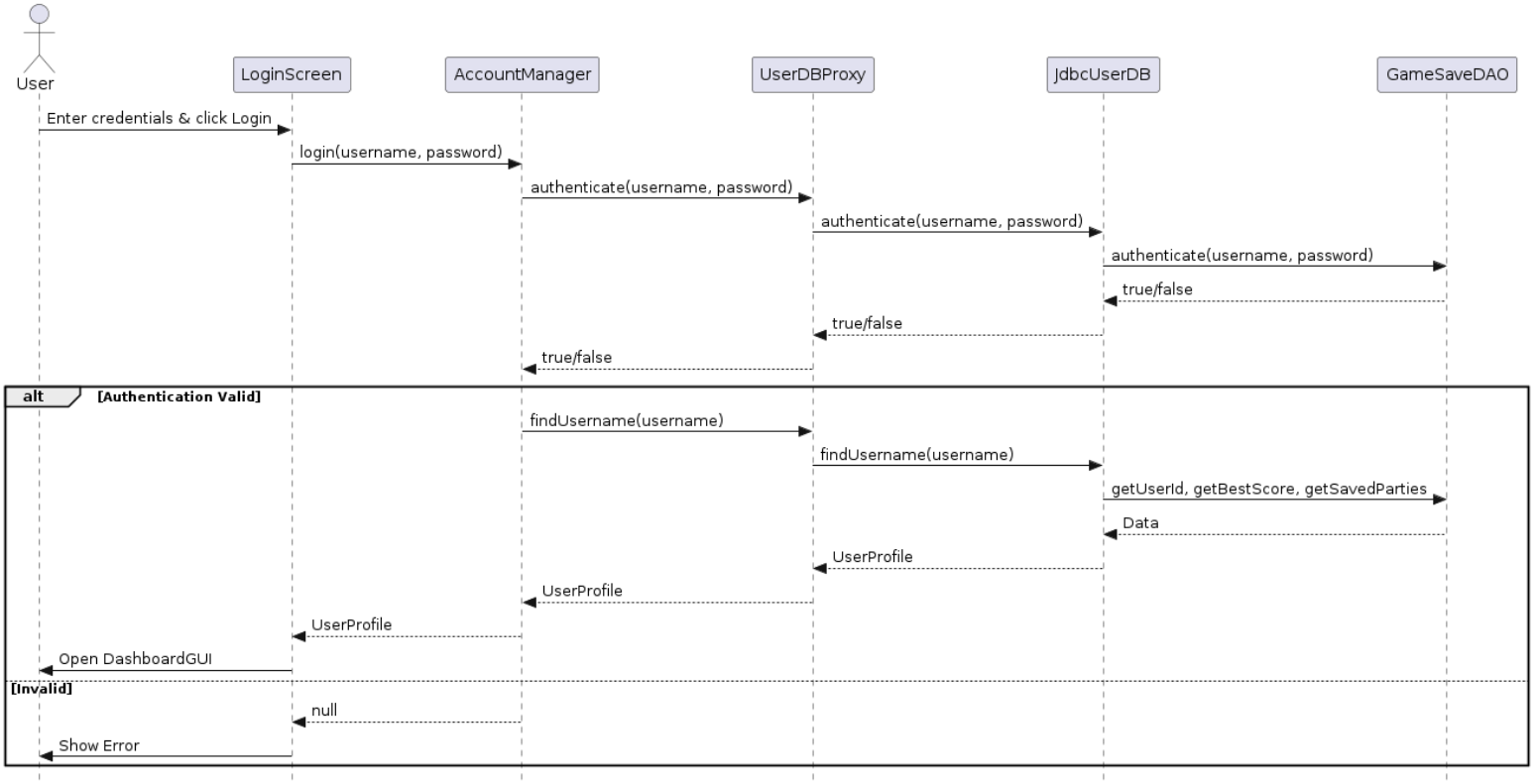


```yaml
services:
  mysql:
    image: mysql:8.0
    container_name: legends-mysql
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: losw
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "--proot"]
      interval: 10s
      timeout: 5s
      retries: 10

  data-service:
    build: ./data-service
    container_name: data-service
    ports:
      - "5003:5003"
    environment:
      DB_URL: jdbc:mysql://mysql:3306/losw
      DB_USER: root
      DB_PASSWORD: root
    depends_on:
      mysql:
        condition: service_healthy

  profile-service:
    build: ./profile-service
    container_name: profile-service
    ports:
      - "5000:5000"
    environment:
      DATA_SERVICE_URL: http://data-service:5003
    depends_on:
      - data-service

  battle-service:
    build: ./battle-service
    container_name: battle-service
    ports:
      - "5001:5001"

  pve-service:
    build: ./pve-service
    container_name: pve-service
    ports:
      - "5002:5002"
    environment:
      DATA_SERVICE_URL: http://data-service:5003
      BATTLE_SERVICE_URL: http://battle-service:5001
    depends_on:
      - data-service
      - battle-service

  pvp-service:
    build: ./pvp-service
    container_name: pvp-service
    ports:
      - "5004:5004"
    environment:
      DATA_SERVICE_URL: http://data-service:5003
      BATTLE_SERVICE_URL: http://battle-service:5001
    depends_on:
      - data-service
      - battle-service

volumes:
  mysql_data:
```

MySQL is the first to start and it must pass its health check before Data Service can start. Data Service has to be running before Profile, PvE and PvP are allowed to start as they all rely on it. Once all services are running, gateway is the last one to start. This order ensures that no service tries to connect to another before it is ready and each service contains its own health check so Docker is able to monitor if it is running correctly.

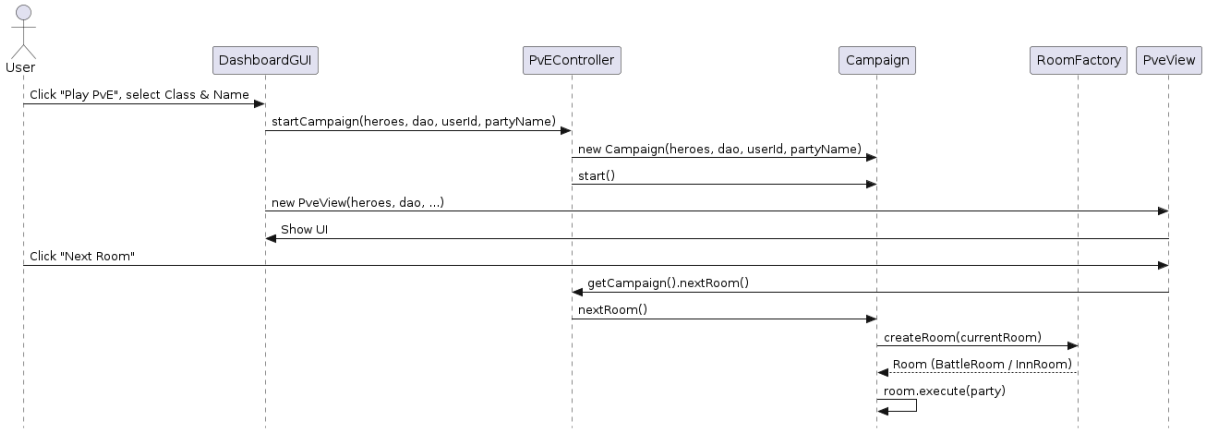
Sequence Diagrams

USE CASE 1: Profile creation and log in



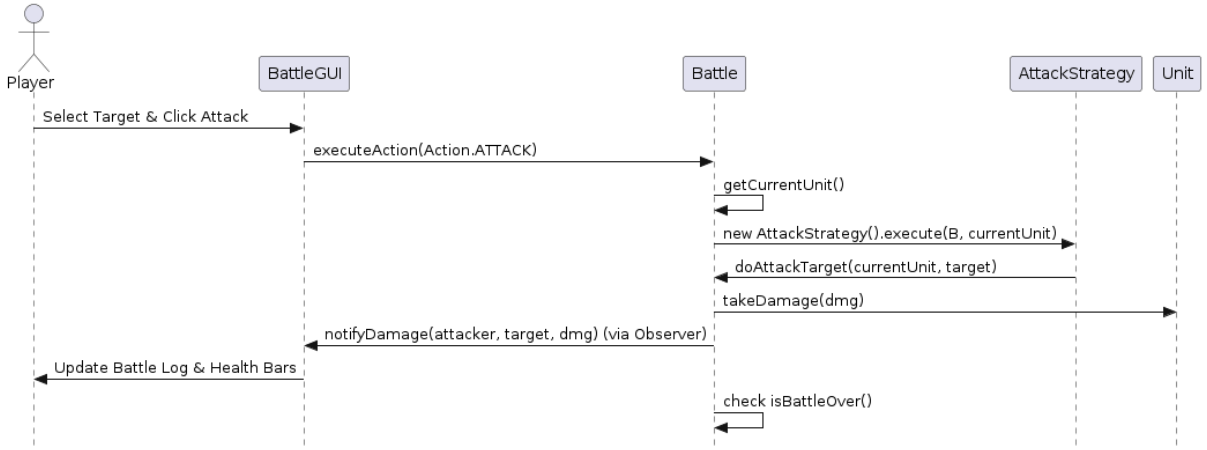
The user inputs their username and password on the LoginScreen. The request traverses the AccountManager to a UserDBProxy, which forwards it to the JdbcUserDB. The GameSaveDAO evaluates the credentials matching the hashed password in the MySQL database. On success, the user profile is assembled and the DashboardGUI opens.

USE CASE 2: Pve Campaign



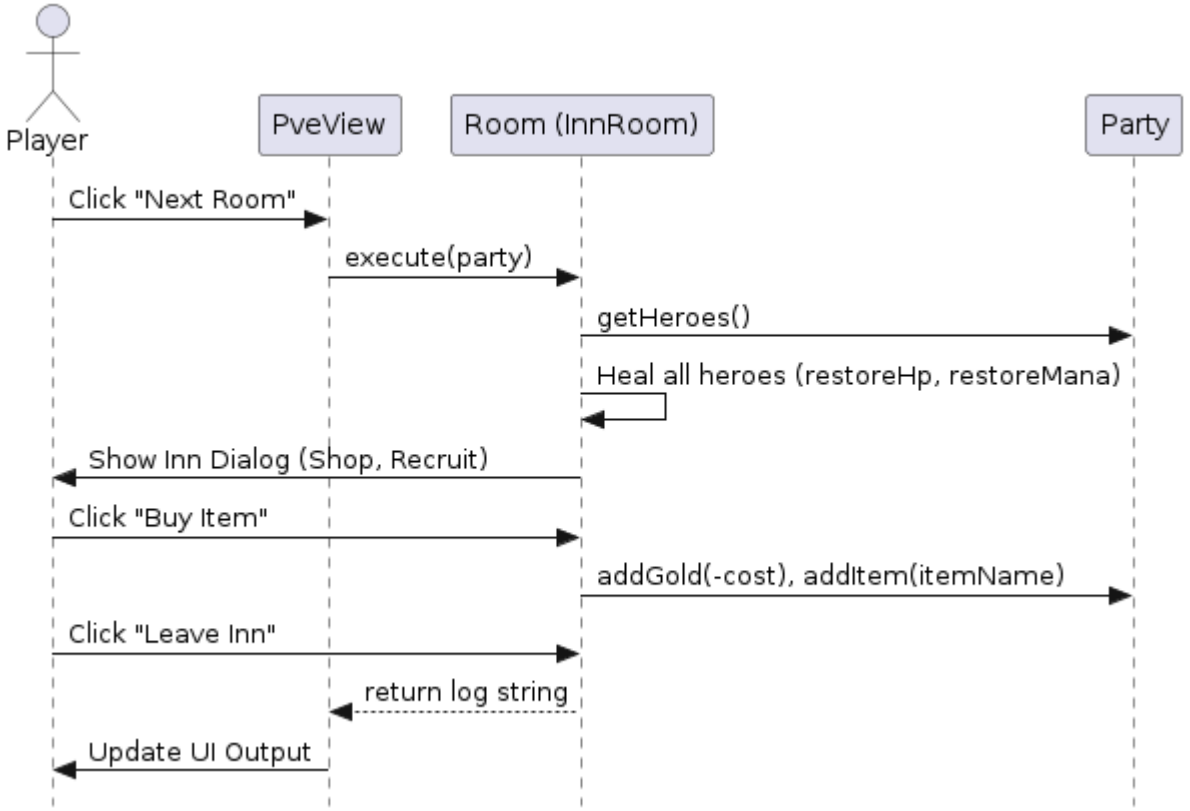
Launching a new PvE campaign initializes the PvEController and Campaign model with the user's chosen starting hero. The Dashboard GUI instantiates the PveView. Progression occurs when the user clicks "Next Room", invoking the Campaign to request a new room from the RoomFactory which executes the room logic.

USE CASE 3: Go through a single battle



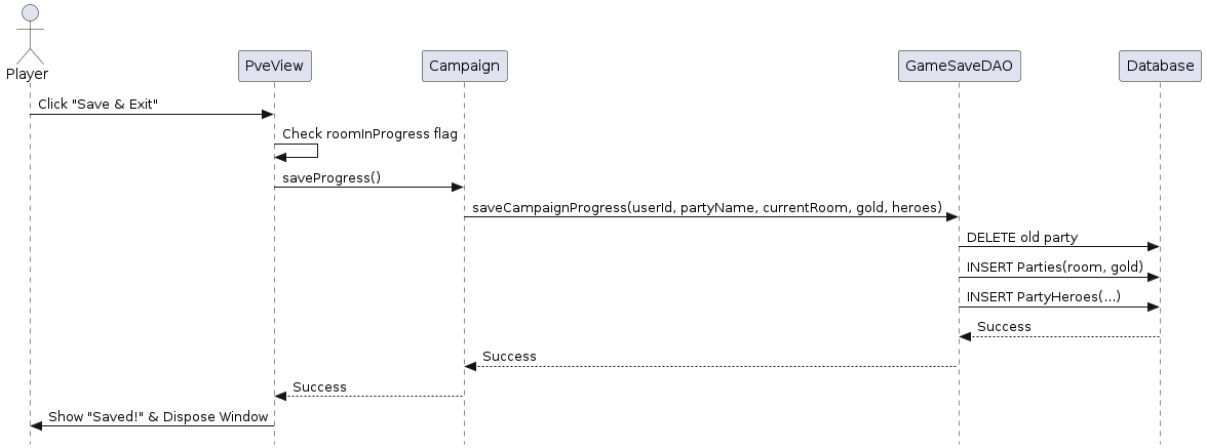
In a battle, the player chooses a target and clicks an action. The BattleGUI delegates this action to the Battle class. The Battle class resolves the action via a Strategy pattern (e.g., AttackStrategy). Damage is applied to the Unit, and the Battle object notifies observers (BattleGUI) to update the visual log and health indicators.

USE CASE 4: The Inn



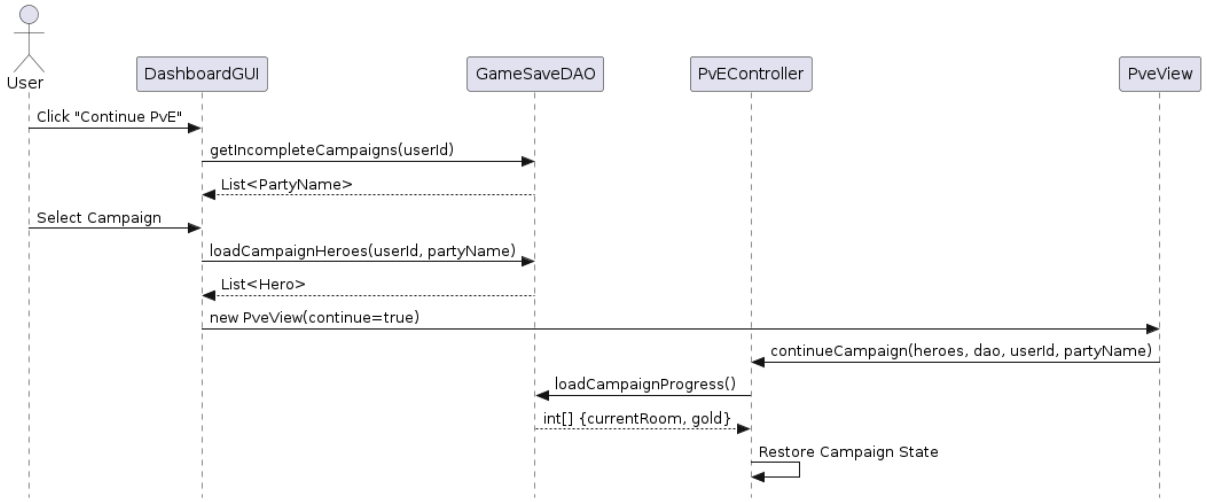
Entering an InnRoom triggers the execute() method via a background worker thread. The Inn immediately restores HP and Mana for all party heroes. A modal dialog presents options to buy items or recruit heroes. Interactions directly modify the Party's gold and inventory states. Leaving the dialog unblocks the thread and returns execution control to the PveView.

USE CASE 5: Exiting and Saving the PvE Campaign



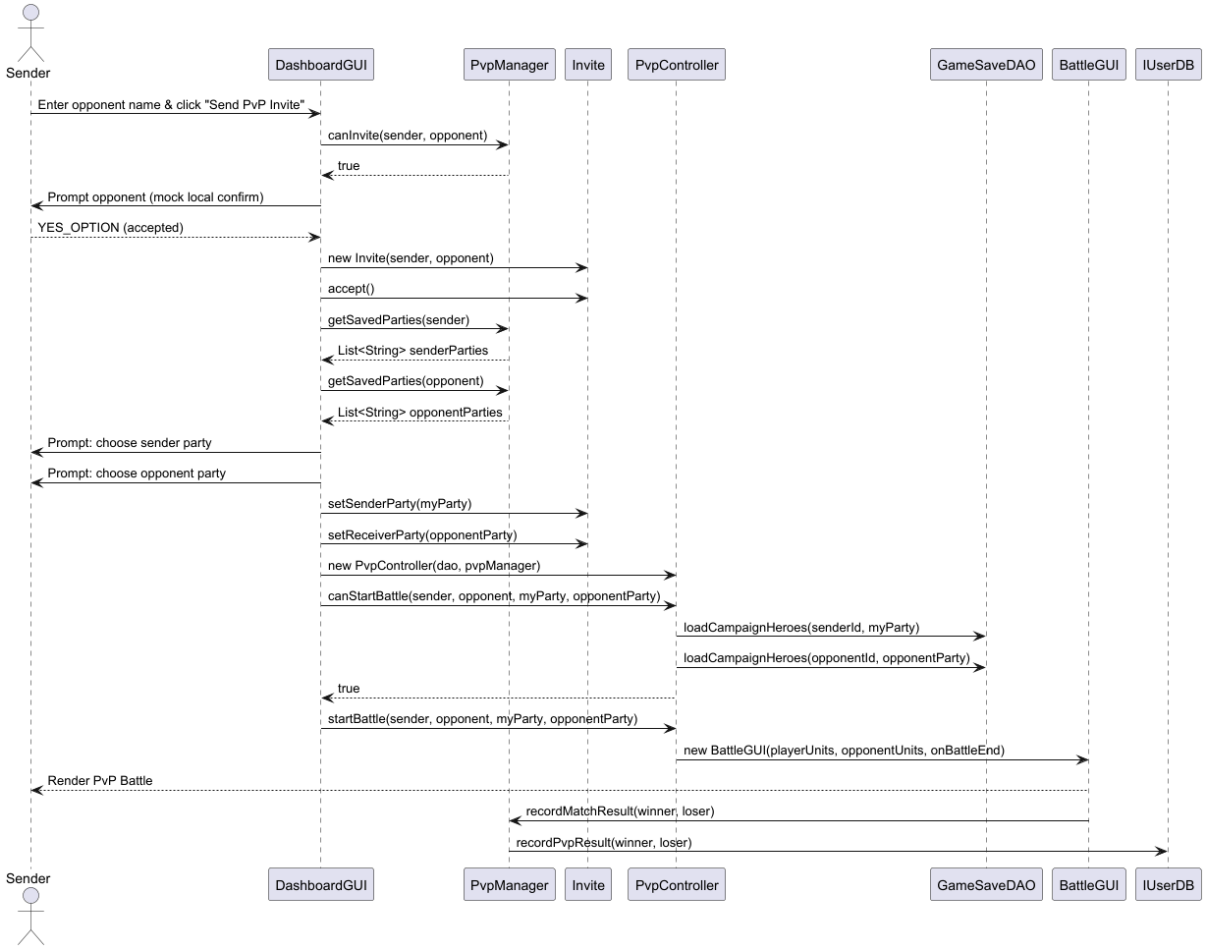
When the player decides to exit the campaign, PveView delegates the save request to the Campaign instance. The Campaign instructs GameSaveDAO to serialize system state, wiping out the former party entries for the given slot and inserting the current states (gold, specific room, deeply-saved heroes) into the relational database.

Use Case 6: Continue an incomplete PvE campaign



The system queries GameSaveDAO for campaigns yet to reach the final room. The user chooses one, and the Dashboard reconstructs the hero objects via DAOs. It forwards these objects alongside the party name to PveView, which asks PvEController to mount the exact state array variables (gold, room number) obtained from the DB.

Use Case 7: PvP Invitation



The user enters an opponent username and clicks the button “Send PvP Invite” which then DashboardGUI calls PvpManager.canInvite() to ensure both players meet the PvP requirements. A message pops up to accept the invite which creates the Invite object and makes the players select a saved party. Then, DashboardGUI creates a PvpController and calls canStartBattle() to load the heroes in the players’ parties from GameSaveDAO to check if they meet the criteria. If so, startBattle() is called launching BattleGUI and both players play the battle and the result is recorded.

Architecture

Profile Creation System Modules			
Module Name	Description	Exposed Interface Names	Interface Description
M1: UI Module	Handles user interaction such as entering information, button interaction, displays success and error messages, and navigation to the dashboard.	M1:IAuthUI	Methods for user actions involving registration and login
M2: Profile Logic	Contains logic for checking register and login validity.	M2:IAccountService	Methods for account creation and checking login information.
M3: Data Access	Communicates with database to store and get information	M3:IUserDB	Methods for checking username existence, saving user account information and retrieving user data.
M4: Database	Takes care of database connection, executes queries to permanently store data.	M4:IDBConnection	Methods for opening database connection and executing queries.

Profile Creation System Interfaces		
Interface Name	Operations	Operation Descriptions
LoginScreen	void register(String username, String password) used by M2	LoginScreen:register(String username, String password) - sends user registration input to account logic.

	void login(String username, String password) used by M2	LoginScreen:login(String username, String password) - sends user login input to account logic.
AccountManager	boolean register(String username, String password) used by M1 boolean login(String username, String password) used by M1	AccountManager:register(String username, String password) - validates and creates a new user if the username is available AccountManager:login(String username, String password) - validates user information and returns if login is successful
IUserDB	boolean usernameExists(String username) used by M2 void save(UserProfile user) used by M2 UserProfile findUsername(String username) used by M2	IUserDB:usernameExists(String username) - checks if a username already exists in the database. IUserDB:createUser(String username, String password) - stores a new user profile in the database. IUserDB:findUsername(String username) - retrieves a stored user profile by username.
DatabaseConnector	void openConnection() used by M3 Connection getConnection() used by M3	DatabaseConnector:openConnection() - establishes a connection to the database. DatabaseConnector:getConnection() - returns active database connection

DataBase Modules and Interface

Database Modules			
Module Name	**Description**	**Exposed Interface Names**	**Interface Description**
PersistenceModule	Handles all interactions with	PM:IGameSave	PM:IGameSave: Provides operations to

	the MySQL database, ensuring that game states, party status, and campaign progress are securely stored and retrieved without exposing SQL logic to the rest of the application.		save the current state of a PvE campaign, including individual hero statistics.

Database Interfaces		
Interface Name	**Operations**	**Operation Descriptions**
GameSaveDAO	saveCampaignProgress(int userId, String partyName, int currentRoom, int gold, List<Hero> heroes, Map<String, Integer> inventory) used by CampaignManager and PvEModule.	GameSaveDAO:saveCampaignProgress(...): Connects to the database and stores the party's current room, gold, and individual hero stats (HP, Mana, Attack, Defense) using batch SQL statements.
GameSaveDAO	loadCampaignHeroes(int userId, String partyName). loadCampaignProgress(int userId, String partyName)	PM:IGameSave:fetchSavedCampaign(...): Queries the database to retrieve a user's active campaign, reconstructing the Party and Hero objects to resume the game.

The PersistenceModule acts as the dedicated bridge between the game's core logic and the MySQL database. It is designed to maintain strict boundaries and low coupling; it interacts primarily with the system controllers (such as the PveController and CampaignManager) without exposing any underlying SQL code to them. When a user saves or continues a campaign, the game controllers call the PM:IGameSave interface. The PersistenceModule then executes the necessary database queries, seamlessly translating Java Party and Unit objects into relational data, and vice versa.

PvE System Modules

Module Name	**Module Name**	**Exposed Interface Names**	**Interface Description**
M1: PvE Controller Module	Manages the flow of the campaign, including starting new campaigns, continuing saved campaigns, and retrieving the current campaign.	PvEController	Methods to start campaigns, continue saved campaigns, and get the current campaign.
M2: Campaign Manager	Represents the campaign state: current room, party, score calculation, saving progress, and finishing the campaign.	Campaign	Methods to get/set current room, gold, last inn room, calculate score, save progress, and retrieve party.
M3: Room Factory	Creates rooms (BattleRoom or InnRoom) dynamically based on floor and party level.	RoomFactory	Method to generate a room: createRoom(floor, totalPartyLevel)
M4: Room	Abstract superclass for all room types; defines the contract for executing a room.	Room	Method execute(Party party) that runs the room logic and returns a result string.
M5: Battle Room	Implements battles: generates enemies, launches GUI, calculates victory/defeat rewards and penalties.	BattleRoom	Method to execute battle, handle results, update party gold/EXP.

	.		
M6: Inn Room	Implements healing, item shop, and hero recruitment; interacts with Swing GUI.	InnRoom	Method to execute roo recruitment.
M7: Party & Hero	Encapsulates the party and heroes, including inventory, gold, health, mana, and experience.	Party / Hero	Methods for adding gold, items, heroes, restoring HP/mana, leveling, and experience gain.
M8: Database / DAO	Saves and loads campaign progress, inventory, and scores.	GameSaveDAO	Methods for saving campaign progress, inventory, and scores; loading state and inventory.

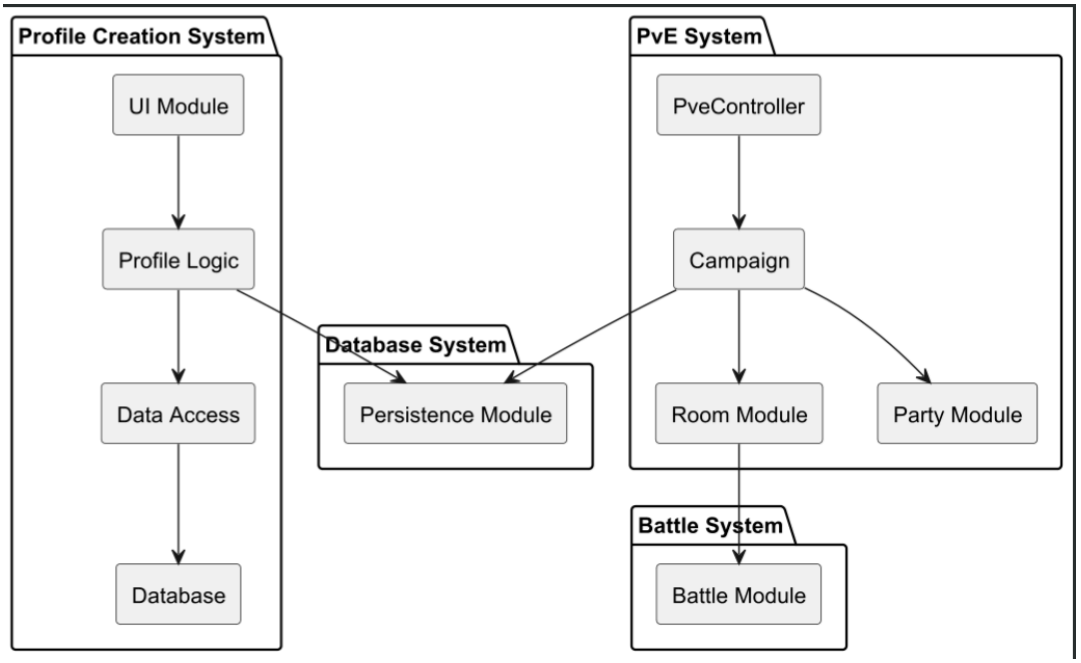
Interface Name	**Operations**	**Operation Description**
IPvEControl	startCampaign(List<Hero> heroes) startCampaign(List<Hero> heroes, GameSaveDAO dao, int userId, String partyName) continueCampaign(List<Hero> heroes, GameSaveDAO dao, int userId, String partyName) getCampaign()	• **startCampaign(List <Hero> heroes)** – Starts a new campaign with the given heroes. • **startCampaign(List <Hero> heroes, GameSaveDAO dao, int userId, String partyName**) – Starts a new campaign with database save integration.

		• **continueCampaign(List<Hero> heroes, GameSaveDAO dao, int userId, String partyName)** – Loads a saved campaign from the database and continues it. • **getCampaign()** **–** Returns the currently active campaign instance.
ICampaign	start() nextRoom() getParty() saveProgress()	• **start()** – Initializes campaign progression and displays a welcome message. • **nextRoom()** – Advances to the next room, executes its logic, and returns the results as a string. • **getParty()** – Returns the Party object associated with the campaign. • **saveProgress()** – Saves the campaign's current state, inventory, and gold to the database.
IRoom	execute(Party party) getFloor()	• **execute(Party party)**

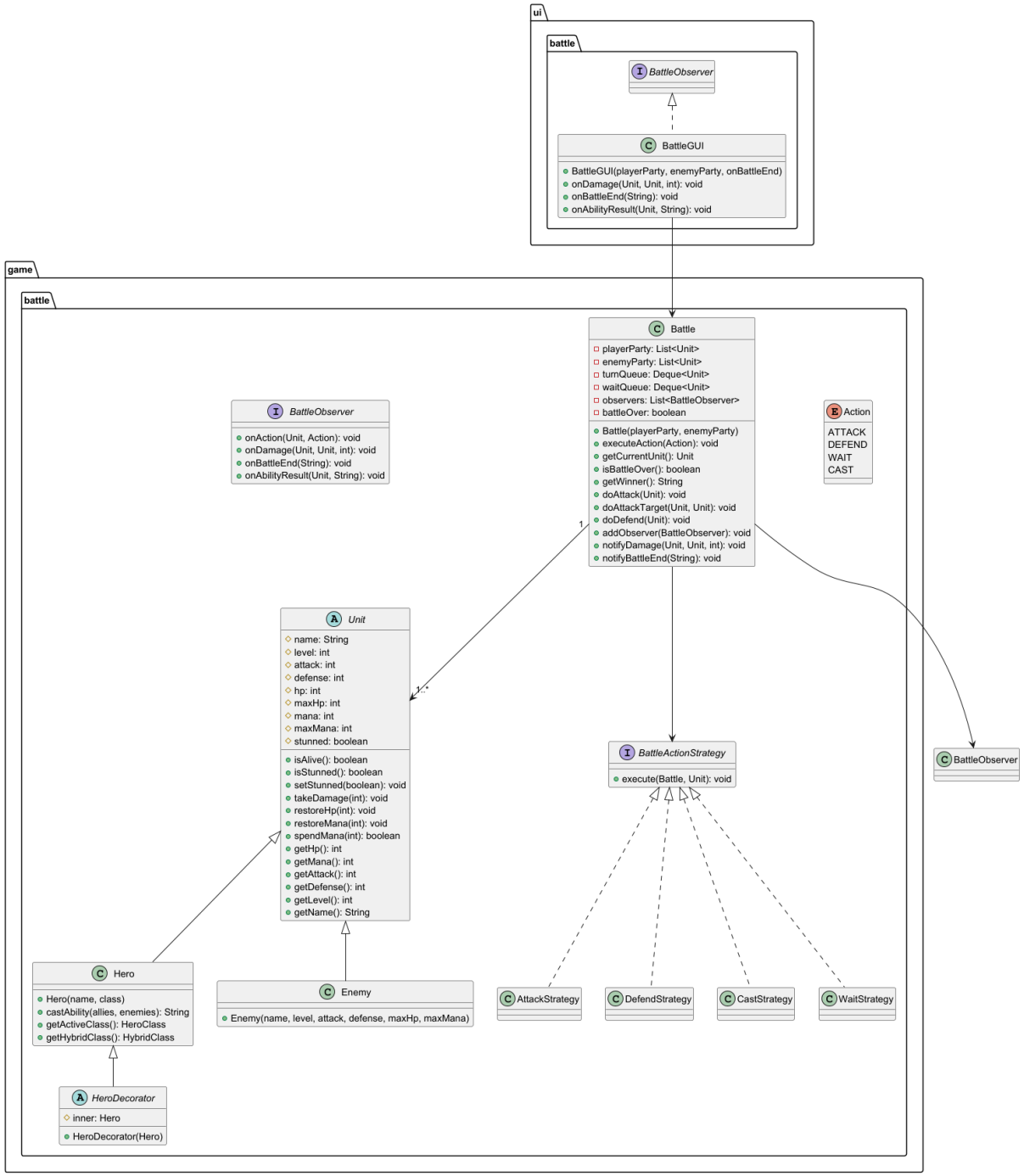
		– Executes room-specific behavior when a party enters. For BattleRoom or InnRoom, this may block until user interaction completes. • **getFloor()** – Returns the floor number of the room.
Party	addHero(Hero hero) getHeroes() addGold(int amount) addItem(String itemName, int quantity)	• **addHero(Hero hero)** – Adds a hero to the party. • **getHeroes()** – Returns a list of all heroes in the party. • **addGold(int amount)** – Adds or subtracts gold from the party. • **addItem(String itemName, int quantity)** – Adds the specified quantity of an item to the party's inventory.

The PvE Controller Module acts as the entry point for PvE mode. It receives a list of heroes and constructs a Party object. It then initializes the Campaign module. The Campaign Module maintains the state of the dungeon, including the current room and the associated party. It provides controlled access to the party through the getParty() method.

The Room Interaction Module defines an abstract Room class that declares the enter(Party) method. Concrete implementations such as BattleRoom and InnRoom extend Room and provide specialized behavior when a party enters them. The Party Management Module encapsulates the list of Hero objects and provides controlled operations to modify or retrieve the hero list.



Class Diagrams and Initial Implementation

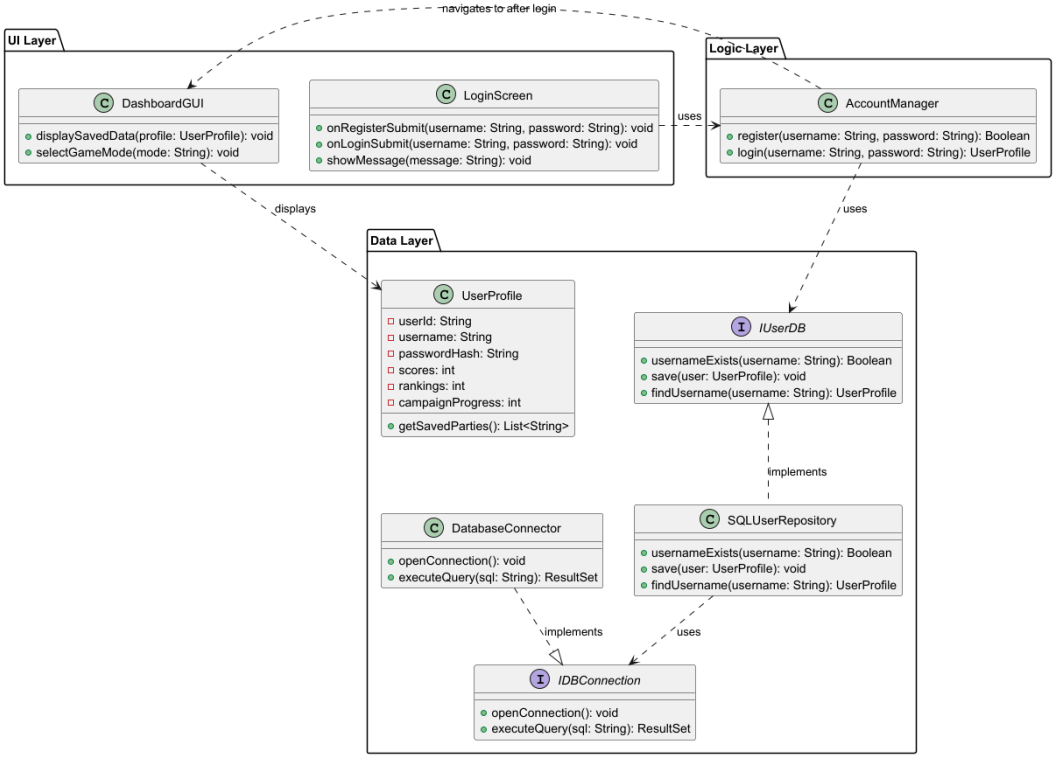


Class name	Attribute/Method name	Description
Unit	Name: String	Name of the unit
Unit	level: int	Current player level
Unit	attack: int	Attack power.
Unit	defense: int	Damage reduction
Unit	hp: int	Current health points
Unit	maxHp: int	Maximum health points of a hero/enemy
Unit	mana: int	Current mana points
Unit	maxMana: int	Maximum mana points
Unit	stunned: boolean	Stun status for whether the unit skips their next turn
Unit	isAlive(): boolean	Returns true if hp > 0
Unit	takeDamage(int): void	Reduces health points by damage
Unit	restoreHp(int amount): void	Increases health points up to max health points
Unit	restoreMana(int amount): void	Increases mana up to maxMana
Battle	enemyParty: List<Unit>	Collection of enemy units
Battle	turnQueue: Queue<Unit>	Queue for turn order
Battle	waitQueue: Queue<Unit>	Queue for units that chose WAIT
Battle	observers:List<BattleObserver>	Observers notified of battle events
Battle	battleOver: boolean	Indicates if battle ended

Battle	executeAction(Action): void	Executes the given action for current unit
Battle	getCurrentUnit(): Unit	Returns the unit whose at turn
Battle	isBattleOver(): boolean	Checks if one side has no more units alive
Battle	getWinner(): String	Returns winning party
Battle	doAttack(Unit): void	Executes an attack on lowest-HP target
Battle	doAttackTarget(Unit, Unit): void	Attack on specific target
Battle	doDefend(Unit): void	Restores 10 HP and 5 Mana
Battle	addObserver(BattleObserver): void	Registers new observer
Battle	notifyDamage(Unit, Unit, Int): void	Tells observers when damage is done
Battle	notifyBattleEnd(String): void	Tells observers when battle ends
Hero	Hero(name, class)	Creates hero with starting class
Hero	castAbility(allies, enemies): String	Performs hero’s class ability
Hero	getActiveClass(): HeroClass	Returns hero’s current active class
Enemy	Enemy(name, level, attack, defense, maxHp, maxMana)	Creates enemy unit
Action	ATTACK, DEFEND, WAIT, CAST	Actions that can be done during a turn
BattleActionStrategy	execute(Battle, Unit): void	Interface defining a single executable action
AttackStrategy	execute(Battle, Unit): void	Attacks lowest-hp enemy or a selected target
DefendStrategy	execute(Battle, Unit): void	Restores hp and mana
CastStrategy	execute(Battle, Unit): void	Performs class’ ability

WaitStrategy	execute(Battle, Unit): void	Passes turn moving it to end of queue
HeroDecorator	Inner:Hero	Wrapped Hero whose stats and state are assigned to
HeroDecorator	HeroDecorator(Hero)	Wraps a hero to allow dynamic stat modification
BattleObserver	onDamage(Unit, Unit, int): void	Called when damage is dealt in battle
BattleObserver	onTurnStart(Unit): void	Called at the start of a unit's turn
BattleObserver	onAction(Unit, Action): void	Called when a unit executes an action
BattleObserver	onBattleEnd(String): void	Called when battle ends
BattleObserver	onAbilityResult(Unit, String): void	Called when ability produces a result
BattleGUI	BattleGUI(playerParty, enemyParty, onBattleEnd)	Constructs and renders battle interface
BattleGUI	onAbilityResult(Unit, String): void	Updates battle log with ability result

Profile Creation and Login Class Diagram



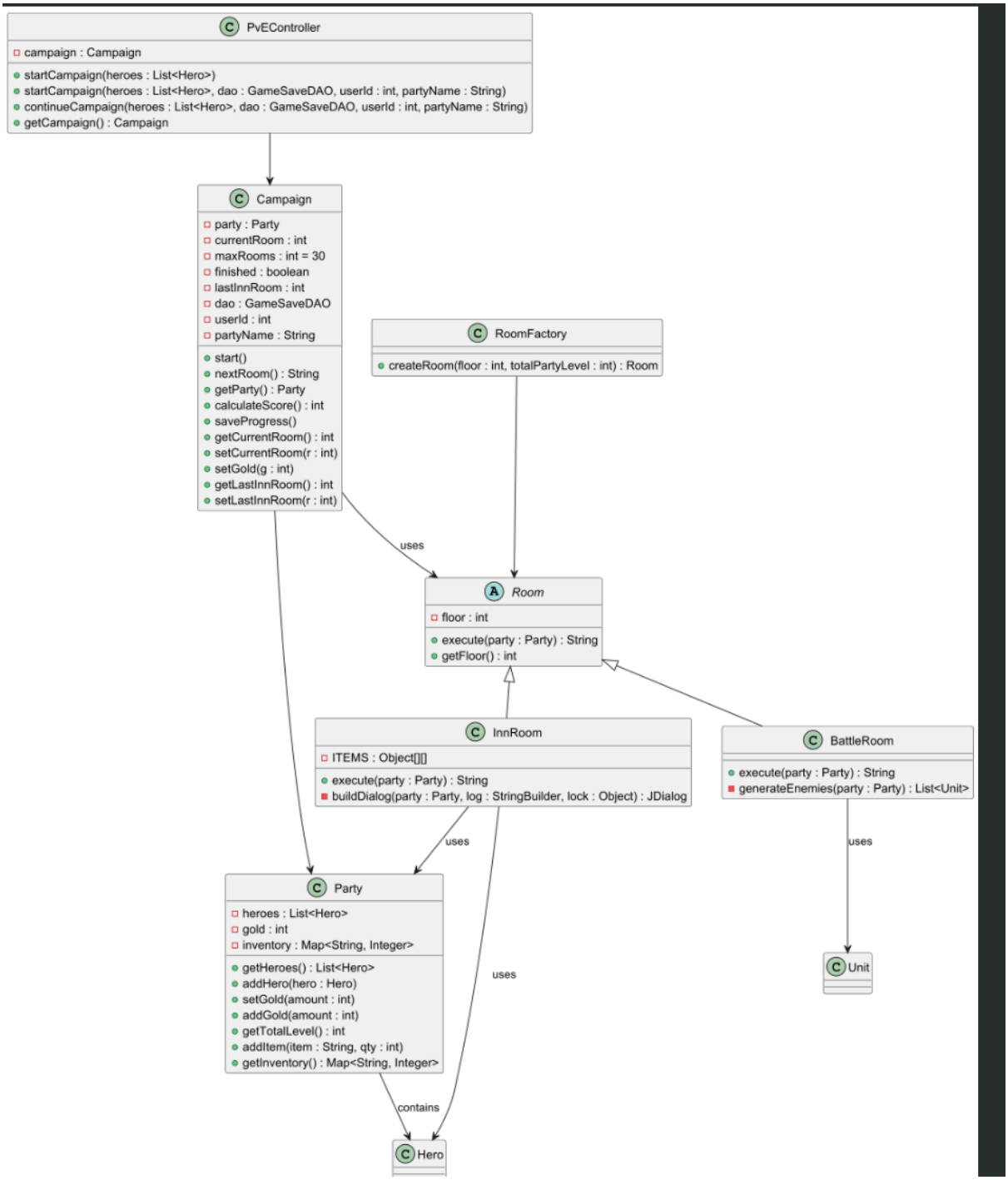
PROFILE CREATION AND LOGIN		
Class Name	Attribute/Method Name	Description
LoginScreen	passwordField: JPasswordField	Input field for the password
LoginScreen	accountManager: AccountManager	Handles registration and login logic
LoginScreen	LoginScreen(AccountManager)	Constructs the login window and wires up button listeners
LoginScreen	showMessage(String): void	Displays a popup message dialog for success or failure outcomes
AccountManager	userDB: IUserDB	The IUserDB instance used to communicate with the database
AccountManager	AccountManager(IUserDB)	Constructs the manager with the given IUserDB implementation
AccountManager	register(String, String): boolean	Validates input and creates a new account if the username is not taken
AccountManager	login(String, String): UserProfile	Verifies credentials and returns the UserProfile if successful, null otherwise

AccountManager	getUserDB(): IUserDB	Returns the IUserDB instance for use by other components such as PvpManager
IUserDB	usernameExists(String): boolean	Checks if the given username already exists in the database
IUserDB	createUser(String, String): void	Stores a new user in the database
IUserDB	authenticate(String, String): boolean	Verifies a username and password against stored credentials
IUserDB	findUsername(String): UserProfile	Retrieves a stored UserProfile by username
IUserDB	hasSavedParty(String): boolean	Returns whether the user has at least one saved party
IUserDB	getSavedParties(String): List<String>	Returns all saved party names for the user
IUserDB	recordPvpResult(String, String): void	Records the outcome of a PvP match
IUserDB	findUsername(String): UserProfile	Retrieves a stored UserProfile by username
UserDBProxy	realDB: IUserDB	The real IUserDB implementation being wrapped

UserDBProxy	UserDBProxy(IUserDB)	Wraps a real IUserDB to provide an interception point without modifying the underlying implementation
UserDBProxy	All IUserDB methods	Delegates every call directly to the wrapped realDB instance
JdbcUserDB	dao: GameSaveDAO	The DAO used to execute all SQL operations
JdbcUserDB	JdbcUserDB(GameSaveDAO)	Constructs the DB implementation with a DAO for SQL operations
JdbcUserDB	usernameExists(String): boolean	Checks if a username exists via GameSaveDAO.getUserIdByUsername()
JdbcUserDB	createUser(String, String): void	Creates a new user by delegating to GameSaveDAO.createUser()
JdbcUserDB	authenticate(String, String): boolean	Delegates credential verification to GameSaveDAO.authenticate()
JdbcUserDB	findUsername(String): UserProfile	Builds and returns a UserProfile by loading ID, scores, saved parties, and campaign progress from GameSaveDAO

UserProfile	userId: int	Unique integer ID for the user account
UserProfile	username: String	Username for login and display
UserProfile	scores: int	The user's best campaign score
UserProfile	campaignProgress: int	The furthest room reached across all campaigns
UserProfile	savedParties: List<String>	List of saved party names for the user
UserProfile	getUserId(): int	Returns the user's ID
UserProfile	getUsername(): String	Returns the username
UserProfile	getScores(): int	Returns the best score
UserProfile	setScores(int): void	Sets the best score
UserProfile	getCampaignProgress(): int	Returns the campaign progress room number
UserProfile	setCampaignProgress(int): void	Sets the campaign progress room number
UserProfile	getSavedParties(): List<String>	Returns the list of saved party names

UserProfile	setSavedParties(List<String>): void	Sets the list of saved party names
DatabaseConnector	instance: DatabaseConnector	The single shared instance (Singleton)
DatabaseConnector	getInstance(): DatabaseConnector	Returns the singleton instance, creating it if necessary
DatabaseConnector	getConnection(): Connection	Opens and returns a new JDBC connection to the database


PvEController
campaign : Campaign
startCampaign(heroes : List<Hero>)
startCampaign(heroes : List<Hero>, dao : GameSaveDAO, userId : int, partyName : String)
continueCampaign(heroes : List<Hero>, dao : GameSaveDAO, userId : int, partyName : String)
getCampaign() : Campaign
Campaign
party : Party
currentRoom : int
maxRooms : int = 30
finished : boolean
lastInnRoom : int
dao : GameSaveDAO
userId : int
partyName : String
start()
nextRoom() : String
getParty() : Party
calculateScore() : int
saveProgress()
getCurrentRoom() : int
setCurrentRoom(r : int)
setGold(g : int)
getLastInnRoom() : int
setLastInnRoom(r : int)
RoomFactory
createRoom(floor : int, totalPartyLevel : int) : Room
uses
Room
floor : int
execute(party : Party) : String
getFloor() : int
InnRoom
ITEMS : Object[][]
execute(party : Party) : String
buildDialog(party : Party, log : StringBuilder, lock : Object) : JDialog
BattleRoom
execute(party : Party) : String
generateEnemies(party : Party) : List<Unit>
uses
uses
Party
heroes : List<Hero>
gold : int
inventory : Map<String, Integer>
getHeroes() : List<Hero>
addHero(hero : Hero)
setGold(amount : int)
addGold(amount : int)
getTotalLevel() : int
addItem(item : String, qty : int)
getInventory() : Map<String, Integer>
uses
Unit
contains
Hero

Class Name	Attribute/Method Name	Description
PvEController	campaign : Campaign	Holds the active campaign instance and acts as the entry point for PvE mode.
PvEController	startCampaign(heroes : List<Hero>)	Initializes a new campaign with the given heroes.
PvEController	startCampaign(heroes : List<Hero>, dao : GameSaveDAO, userId : int, partyName : String)	Starts a campaign with saved data integration.
PvEController	continueCampaign(heroes : List<Hero>, dao : GameSaveDAO, userId : int, partyName : String)	Resumes a previously saved campaign, restoring party and inventory.
PvEController	getCampaign() : Campaign	Returns the current active campaign.
Campaign	party : Party	The party associated with this campaign.

Campaign	currentRoom : int	Tracks the current room number.
Campaign	maxRooms : int	Maximum number of rooms i
Campaign	finished : boolean	Indicates if the campaign is complete.

Campaign	lastInnRoom : int	Tracks the last inn visited for hero recovery/recruiting.
Campaign	dao : GameSaveDAO	Database access object for saving/loading campaign data.
Campaign	userId : int	Identifier for the player in the database.
Campaign	partyName : String	Name of the party.
Campaign	start()	Begins the campaign, displa message.
Campaign	nextRoom() : String	Progresses to the next room, behavior.
Campaign	getParty() : Party	Returns the campaign's party.
Campaign	calculateScore() : int	Computes the final score bas gold.
Campaign	saveProgress()	Saves the current campaign database.
Campaign	getCurrentRoom() : int	Returns the current room number.
Campaign	setCurrentRoom(r : int)	Updates the current room sa bounds.

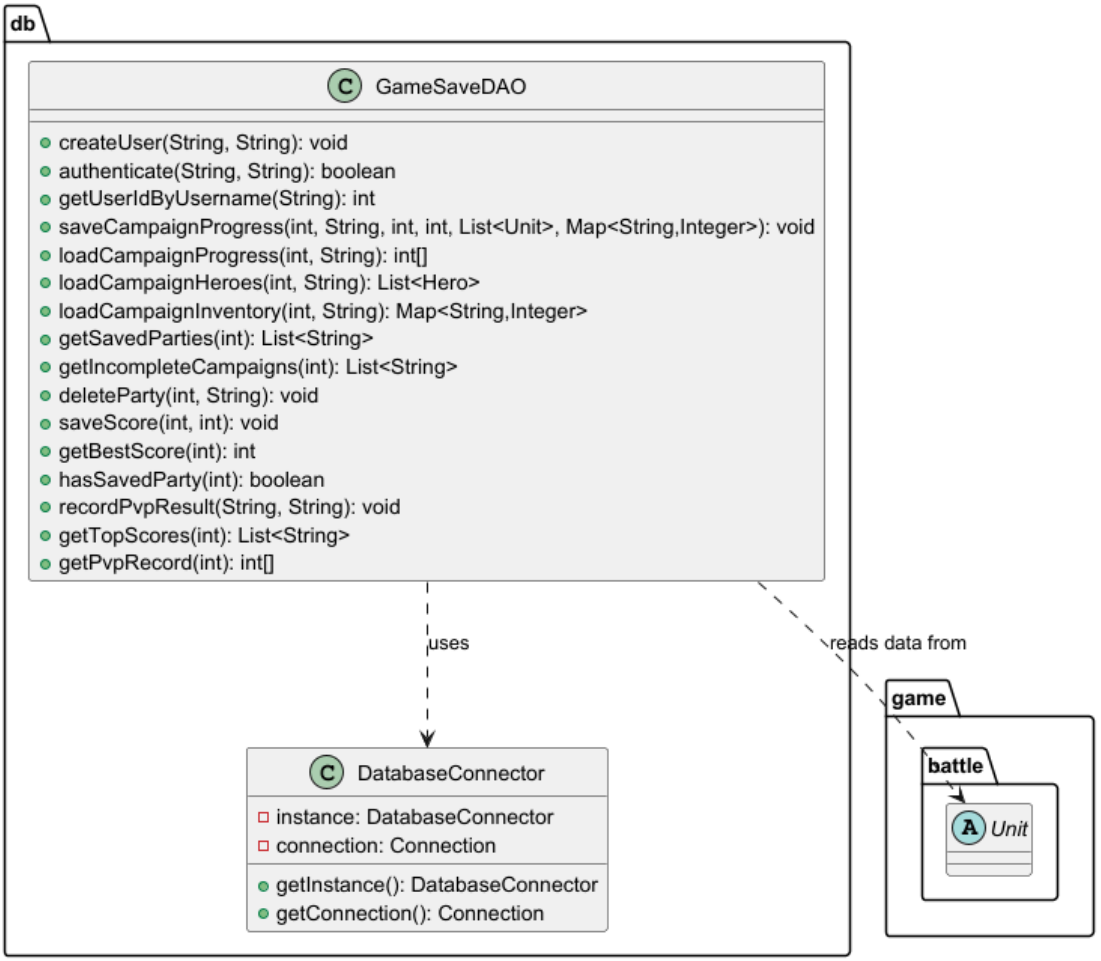
Campaign	setGold(g : int)	Updates party gold.
Campaign	getLastInnRoom() : int	Returns last inn room number.
Campaign	setLastInnRoom(r : int)	Updates last inn room safely.
Room (abstract)	floor : int	Floor number of the room.
Room (abstract)	execute(party : Party) : String	Abstract method; defines room specific behavior when a party enters.
Room (abstract)	getFloor() : int	Returns the floor number.
BattleRoom (extends Room)	execute(party : Party) : String	Generates enemies, runs the battle via BattleGUI, computes rewards or penalties.
BattleRoom (extends Room)	generateEnemies(party : Party) : List<Unit>	Creates a list of enemy units based on party strength.
InnRoom (extends Room)	ITEMS : Object[][]	Static array of items available for purchase or use.
InnRoom (extends Room)	execute(party : Party) : String	Heals all heroes, allows item purchases, recruits heroes, and waits for player interaction.

InnRoom (extends Room)	buildDialog(party : Party, log	Constructs the interactive inn GUI.
RoomFactory	createRoom(floor : int, totalPartyLevel : int) : Room	Dynamically returns either a BattleRoom or InnRoom based on party level and probability.

Party	heroes : List<Hero>	List of heroes in the party.
Party	Gold :int	Amount of gold the party has
Party	inventory : Map<String, Integer>	Tracks items and quantities.
Party	getHeroes() : List<Hero>	Returns the list of heroes.
Party	addHero(hero : Hero)	Adds a hero to the party.
Party	setGold(amount : int)	Sets party gold.
Party	addGold(amount : int)	Modifies party gold.

Party	getTotalLevel() : int	Returns total combined hero levels.
Party	addItem(item : String, qty : in	Adds an item to the inventory.
Party	getInventory() : Map<String, Integer>	Returns inventory map.

Database UML Diagram

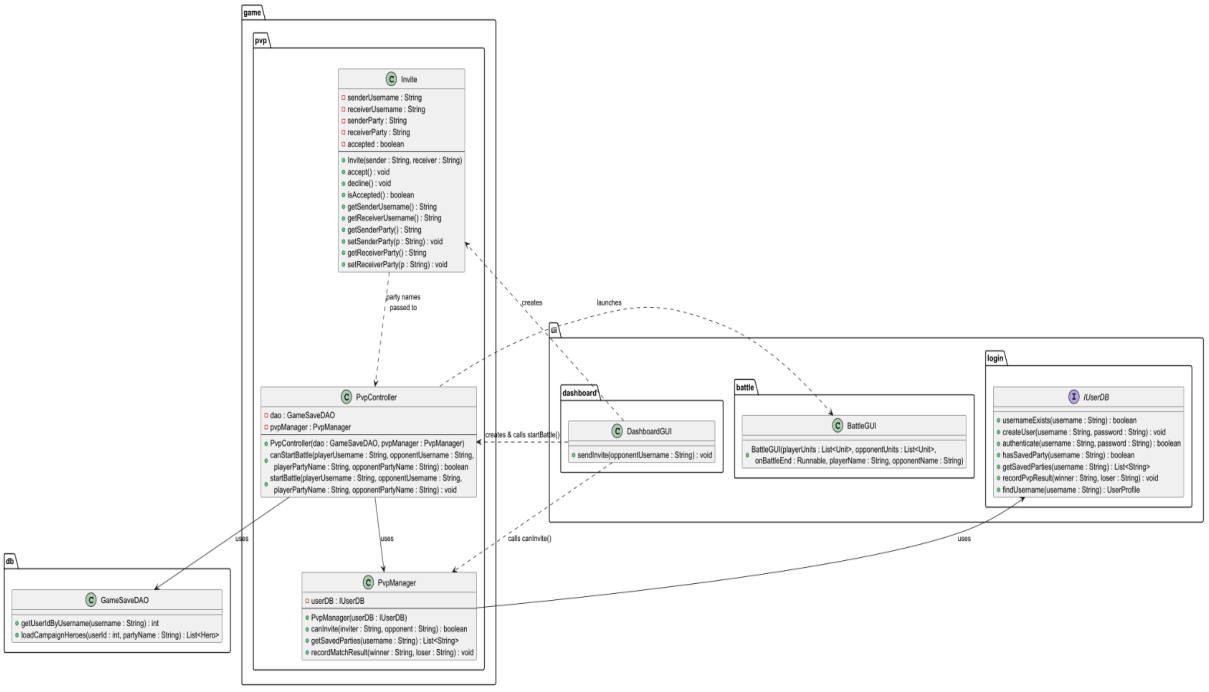


Class Name	Attribute/Method Name	Description
DatabaseConnector	URL: String	The JDBC connection string to the local MySQL database
DatabaseConnector	USER: String	The database username credential
DatabaseConnector	PASS: String	The database password credential
DatabaseConnector	instance: DatabaseConnector	The single shared instance (Singleton pattern)
DatabaseConnector	getInstance(): DatabaseConnector	Returns the singleton instance, creating it if necessary
DatabaseConnector	getConnection(): Connection	Opens and returns a new JDBC connection to the database
GameSaveDAO	createUser(String, String): void	Inserts a new user with a hashed password into the Users table
GameSaveDAO	authenticate(String, String): boolean	Verifies a username and hashed password against the database
GameSaveDAO	getUserIdByUsername(String): int	Returns the user's ID, or -1 if not found

GameSaveDAO	saveCampaignProgress(int, String, int, int, List<Unit>, Map<String,Integer>): void	Deletes the existing party entry and inserts the current room, gold, heroes, and inventory into the database
GameSaveDAO	loadCampaignProgress(int, String): int[]	Returns the current room and gold for a saved party as an int array
GameSaveDAO	loadCampaignHeroes(int, String): List<Hero>	Reconstructs all Hero objects for a saved party from the database
GameSaveDAO	loadCampaignInventory(int, String): Map<String,Integer>	Returns the saved inventory map for a party
GameSaveDAO	getSavedParties(int): List<String>	Returns all party names saved for a given user ID
GameSaveDAO	getIncompleteCampaigns(int) : List<String>	Returns party names where current_room < 30, ordered by most recent
GameSaveDAO	deleteParty(int, String): void	Cascades deletion of a party's inventory, heroes, and party record
GameSaveDAO	saveScore(int, int): void	Inserts a campaign completion score into the Scores table
GameSaveDAO	getBestScore(int): int	Returns the user's highest recorded score

GameSaveDAO	hasSavedParty(int): boolean	Returns whether the user has at least one saved party
GameSaveDAO	recordPvpResult(String, String): void	Records a PvP match result and increments win/loss counts for both players
GameSaveDAO	getTopScores(int): List<String>	Returns the top N username-score pairs across all users for the Hall of Fame
GameSaveDAO	getPvpRecord(int): int[]	Returns the win and loss count for a user as an int array

PvP Class Diagram



Class Name	Attribute/Method Name	Description
PvpManager	userDB: IUserDB	The user database used to validate invites and retrieve saved parties
PvpManager	PvpManager(IUserDB)	Constructs the manager with an IUserDB implementation
PvpManager	canInvite(String sender, String opponent): boolean	Returns true if both players exist, are different users, and each has at least one saved party
PvpManager	getSavedParties(String username): List<String>	Returns the list of saved party names for the given username
PvpManager	recordMatchResult(String winner, String loser): void	Records the PvP match outcome by delegating to IUserDB.recordPvpResult()
PvpController	dao: GameSaveDAO	DAO used to load hero data and record match results
PvpController	pvpManager: PvpManager	Manager used for invite validation and result recording
PvpController	PvpController(GameSaveDAO, PvpManager)	Constructs the controller with a DAO and PvP manager
PvpController	canStartBattle(String playerUsername, String opponentUsername, String	Loads both parties from the database and returns true if both have at least one hero

	playerPartyName, String opponentPartyName): boolean	
PvpController	startBattle(String playerUsername, String opponentUsername, String playerPartyName, String opponentPartyName): void	Loads both parties, creates a Battle instance, launches BattleGUI, and records the result on completion
Invite	senderUsername: String	Username of the player who sent the invite
Invite	receiverUsername: String	Username of the invited player
Invite	senderParty: String	Party name selected by the sender
Invite	receiverParty: String	Party name selected by the receiver
Invite	accepted: boolean	Whether the invite has been accepted
Invite	Invite(String sender, String receiver)	Creates an invite between two players
Invite	accept(): void	Marks the invite as accepted
Invite	decline(): void	Marks the invite as declined
Invite	isAccepted(): boolean	Returns whether the invite was accepted

Invite	getSenderUsername(): String	Returns the sender's username
Invite	getReceiverUsername(): String	Returns the receiver's username
Invite	getSenderParty(): String	Returns the sender's chosen party name
Invite	setSenderParty(String): void	Sets the sender's chosen party name
Invite	getReceiverParty(): String	Returns the receiver's chosen party name
Invite	setReceiverParty(String): void	Sets the receiver's chosen party name

Design Patterns

The PvE module employs several well-known design patterns to achieve modularity, flexibility, and maintainability. The **Factory Pattern** is used in RoomFactory to dynamically create BattleRoom or InnRoom instances based on party level and probability, decoupling object creation from the controller. The PvEController serves as a **Facade**, providing a single entry point for starting or continuing campaigns and hiding the complexity of Campaign, Party, and database interactions. The abstract Room class uses a **Template/Strategy Pattern**, defining the execute() method while allowing subclasses to implement specific room behaviors. Both BattleRoom and InnRoom use a **Callback/Observer Pattern** with Swing GUI interactions and wait/notify to handle asynchronous user input without blocking the controller logic. The Campaign class interacts with GameSaveDAO through the **DAO Pattern**, separating persistence from business logic. Finally, the Party class aggregates multiple Hero objects in a **Composite-like structure**, allowing operations on the party as a whole while managing individual heroes. Together, these patterns ensure high cohesion, low coupling, and a scalable architecture for the PvE system.

Singleton Pattern

The Singleton pattern is implemented in the DatabaseConnector class. This class maintains a single shared instance through a private static variable and provides global access via a getInstance() method. By restricting instantiation, the system ensures that only one database connection manager exists at runtime. This is particularly important for managing JDBC connections efficiently, as it avoids unnecessary duplication of connection objects and ensures consistent interaction with the database across all modules. The use of the Singleton pattern simplifies resource management and prevents potential conflicts arising from multiple concurrent connection instances.

Factory Method Pattern

The Factory Method pattern is used within the RoomFactory class. This class encapsulates the logic required to create different types of rooms, such as BattleRoom and InnRoom, based on parameters like the current floor or party level. Instead of the Campaign class directly instantiating room objects, it delegates this responsibility to the factory. This design centralizes object creation and allows the system to easily extend or modify room generation logic without altering the core campaign flow. As a result, the code remains flexible and adheres to the Open/Closed Principle.

Strategy Pattern

The Strategy pattern is applied in the battle system through the BattleActionStrategy interface and its concrete implementations, such as AttackStrategy, DefendStrategy, CastStrategy, and WaitStrategy. Each strategy encapsulates a specific action that can be performed during a battle. The Battle class dynamically selects and executes these strategies based on the player's chosen action. This eliminates the need for large conditional statements and allows new battle actions to be added without modifying existing logic. The pattern promotes clean separation of behavior and enhances extensibility within the combat system.

Observer Pattern

The Observer pattern is utilized in the interaction between the Battle class and the BattleGUI. The Battle class maintains a list of observers through the BattleObserver interface, and the BattleGUI registers itself as a listener. Whenever a significant event occurs during a battle, such as damage being dealt or an action being executed, the Battle class notifies all registered observers. This allows the GUI to update in real time without being tightly coupled to the battle logic. The Observer pattern effectively separates the game's core logic from its presentation layer, improving modularity and maintainability.

Decorator Pattern

The Decorator pattern is implemented through an abstract class named HeroDecorator. It works by wrapping an existing Hero object and passing all statistics and state calls through to Hero. This allows for temporary bonuses to be applied to a hero during runtime without altering the base Hero class. Any additional effects can be implemented by creating a HeroDecorator subclass which keeps the hero structure untouched.

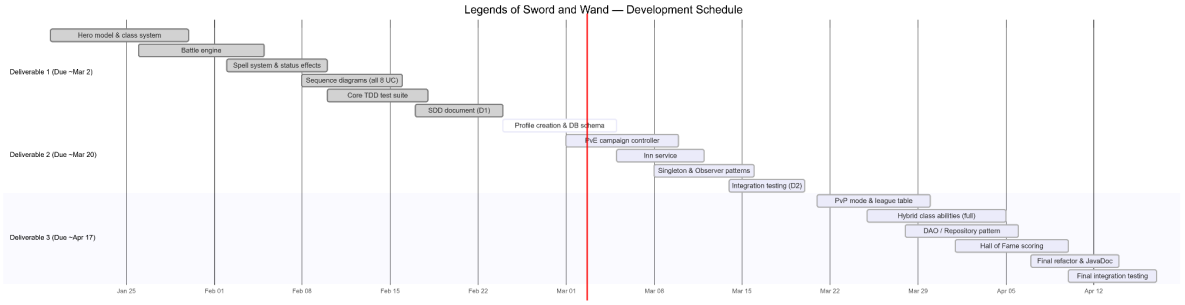
Proxy Pattern

The Proxy pattern is used in the UserDBProxy class, which implements the IUserDB interface and acts as an intermediary between the application and the JdbcUserDB class. Instead of directly interacting with the database implementation, components such as the AccountManager communicate with the proxy. The proxy can control access, perform validation, or add additional behavior such as logging before delegating requests to the real database object. This abstraction enhances security, reduces direct dependency on the database layer, and provides flexibility for future modifications without affecting the rest of the system.

Activities Plan, Product Backlog, and Sprint Backlog

The product backlog includes implementation of advanced combat mechanics, additional room types, hero abilities, item systems and enhanced database features, expansion. Deliverable 1 focuses on foundational systems: profile creation, core battle mechanics, PvE progression, and database integration. Deliverables 2 and 3 will expand functionality, refactor code to include additional design patterns, and improve system robustness.

Gantt Diagram



Product Backlog

ID	Item	Deliverable

PB-01	Hero model + class system	D1 (done)
PB-02	Battle engine (turn-based)	D1 (done)
PB-03	Spell / ability system	D1 (done)
PB-04	Status effects (stun, shield)	D1 (done)
PB-05	Sequence diagrams (all 8 UC)	D1 (done)
PB-06	Core test suite (55 tests)	D1 (done)
PB-07	Profile creation & persistence	D1 (done)
PB-08	PvE campaign controller	D1(done)
PB-10	Database integration (MySQL)	D1 (done)
PB-09	Inn service	D2(done)
PB-11	Singleton pattern (LeagueService, CampaignController)	D2(done)
PB-12	Observer pattern (HP/level-up events)	D2(done)
PB-13	PvP mode + league table	D2(done)
PB-14	Hybrid class ability upgrades (full set)	D2(done)
PB-15	Repository / DAO pattern	D2(done)
PB-16	Hall of Fame scoring	D2(done)
PB-17	Full integration testing	D2(done)
PB-18	Final refactoring & JavaDoc	D2(done)

Group Meeting Logs

Minutes	Attendance	Topics	Decisions	Tasks

45	Everyone	Foundation Role Picks Project plan/ Organization	Decided to use microservice architecture with Spring Boot services	Kevin: Profile, PvP, Data Tobi: PvE, Battle
35	Everyone	Progress Check Group Work session Q&A Research	Confirmed API endpoints for profile service and battle service	Kevin: Implement profile service and database connection. Tobi: Continue battle engine
15	Everyone	Diagram creation Architecture review	Agreed on sequence diagrams for main use cases	Kevin: Profile, data and PvP diagrams. Tobi: PvE and battle diagrams
40	Everyone	Docker and CI/CD	Containerize each service and use Docker Compose	Kevin + Tobi: Implement Dockerfiles and database service
30	Everyone	Review document	Edited and Revised document and services	Kevin + Tobi: Double check document and services
30	Everyone	Deliverable 2 Plan/Work Separation	Gave out tasks to all group members	Kevin + Tobi: Implementing new features
20	Everyone	Progress Check Group Work session	Decided to use the time as a work period	Kevin + Tobi: Worked on Deliverable 2 requirements
35	Everyone	Finalizing Work	Peer reviewed each others work	Kevin + Tobi: Commits
45	Everyone	Wrap Up	Made sure we had everything we needed and discussed any changes	Kevin + Tobi: Updating design document.

Test Driven Development (TDD)

Test ID	PC-TC01
Category	Registration (New user registers)
Requirements Coverage	PC-Successful-Register
Initial Condition	System has been initiated and runs, available database connection and username does not exist.
Procedure	1. User enters username 2. User enters password 3. User clicks register
Expected Outcome	The user's new profile is created and saved in the database then a success message shows up notifying the user.
Notes	User should provide a valid input. Password should be stored as a hash.

Test ID	PC-TC02
Category	Registration (Username exists)

Requirements Coverage	PC-Unsuccessful-Register
Initial Condition	The system has been initiated and runs, available database connection and username does exist.
Procedure	1. User enters username 2. User enters password 3. User clicks register
Expected Outcome	The user receives an error message and registration doesn't continue.
Notes	

Test ID	PC-TC03
Category	Login (Valid Authentication)
Requirements Coverage	PC-Successful-Login
Initial Condition	The system has been initiated and runs, username is stored and exist in the database.

Procedure	1. User enters correct username 2. User enters correct password 3. User clicks login
Expected Outcome	Login is successful and user is transferred to dashboard GUI.
Notes	

Test ID	PC-TC04
Category	Login (Failed Authentication)
Requirements Coverage	PC-Unsuccessful-Login
Initial Condition	The system has been initiated and runs, username is stored and exists in the database.
Procedure	1. User enters correct username 2. User enters incorrect password 3. User clicks login
Expected Outcome	Login is unsuccessful and the user is notified of incorrect password input.

Notes	Dashboard doesn't appear.

Test ID	PC-TC05
Category	Login (Unknown Username)
Requirements Coverage	PC-Unsuccessful-Login
Initial Condition	The system has been initiated and runs. The supplied username does not exist in the database.
Procedure	1. User enters unknown username 2. User enters any password 3. User clicks login
Expected Outcome	Login fails and the error message includes 'not found'.
Notes	Covers the missing-user path.

Test ID	PC-TC06

Category	Registration (Empty Username)
Requirements Coverage	PC-Unsuccessful-Register
Initial Condition	The system has been initiated and runs.
Procedure	1. User leaves username field blank 2. User enters password 3. User clicks register
Expected Outcome	Registration is rejected and the error message includes 'empty'.
Notes	Guards against blank username insertion.

Test ID	PC-TC07
Category	Registration (Password Hashing)
Requirements Coverage	PC-Successful-Register
Initial Condition	The system has been initiated and runs. A new username is used.
Procedure	1. Register a new user with a plain-text password

Expected Outcome	Stored password hash starts with '$2a$' (BCrypt format). Plain-text password is not stored.
Notes	BCrypt hash verified directly in the DB record.

Test ID	PC-TC08
Category	Login (Profile Data Returned)
Requirements Coverage	PC-Successful-Login
Initial Condition	The system has been initiated and runs. A valid user has been registered.
Procedure	1. Register user 2. Login with same credentials
Expected Outcome	Response contains scores=0, rankings=0, campaignProgress=0.
Notes	Validates initial profile field values.

Test ID	DB-TC-01
Category	Database Insertion / Party Status
Requirements Coverage	UC5-Save-Campaign-Progress
Initial Condition	The system is connected to the MySQL database, and User ID 1 exists in the Users table.
Procedure	1. The system creates a list containing two Hero objects (e.g., "Arthur" and "Merlin") with specific HP and Mana values. 2. The system calls saveCampaignProgress(1, "The Round Table", 12, 500, heroes).
Expected Outcome	A new row is added to the Parties table with room 12 and 500 gold. Two new rows are added to the Heroes table, linked to the new party's ID, with exact matching stats.
Notes	Verifies the core requirement of storing party status and campaign progress correctly.

Test ID	DB-TC-02
Category	Database Integrity / Error Handling

Requirements Coverage	UC5-Save-Campaign-Progress
Initial Condition	The system is connected to the MySQL database. User ID 999 does not exist in the Users table.
Procedure	1. The system attempts to save a party by calling saveCampaignProgress(999, "Lost Souls", 5, 100, heroes).
Expected Outcome	The MySQL database rejects the insertion due to a Foreign Key constraint failure. The Java catch block triggers, prints an error message, and no data is written to the Parties or Heroes tables.
Notes	Ensures that orphan records cannot be created without a valid user profile.

Test ID	DB-TC-03
Category	Database Management / Cascading Deletes
Requirements Coverage	UC1-Profile-Management
Initial Condition	User ID 2 exists and has an active saved campaign in the Parties table, along with associated heroes in the Heroes table.

Procedure	1. The system executes a SQL command to delete User ID 2 from the Users table (e.g., simulating account deletion). 2. The system queries the Parties and Heroes tables for records linked to User ID 2.
Expected Outcome	The queries return 0 results. The ON DELETE CASCADE rule from the SQL schema successfully wiped the user's party and hero data automatically.
Notes	Validates the database schema design and ensures the database doesn't fill up with dead data from deleted accounts.

Test ID	DB-TC-04
Category	Database / No Save Returns Empty
Requirements Coverage	UC6-Continue-Campaign-Progress
Initial Condition	The system is running. User ID 999 has no saved campaign.
Procedure	1. Call fetchSavedCampaign(999).
Expected Outcome	The result is empty no exception is thrown.
Notes	Edge case: user has never saved a campaign.

Test ID	DB-TC-05
Category	Database Management / Mark Campaign Inactive
Requirements Coverage	UC5-Save-Campaign-Progress
Initial Condition	User ID 4 has an active saved campaign.
Procedure	1. Save campaign for user 4. 2. Call completeCampaign(4). 3. Call fetchSavedCampaign(4).
Expected Outcome	Completed campaign no longer appears as active fetchSavedCampaign returns empty.
Notes	Prevents completed runs from appearing as continuable saves.

Test ID	DB-TC-06
Category	Database / Hero Stats Persisted Correctly

Requirements Coverage	UC5-Save-Campaign-Progress
Initial Condition	The system is running. User ID 5 saves a campaign with specific hero stats.
Procedure	1. Save campaign with hero Arthur (level=5, attack=20, defense=10). 2. Fetch the saved campaign for user 5.
Expected Outcome	Hero stats match exactly: name=Arthur, level=5, attack=20, defense=10.
Notes	Validates that individual stat columns are correctly written and read.

Test ID	BS-TC-01
Category	Battle System (Action)
Requirements Coverage	UC3-Attack-Damages-HP
Initial Condition	The system is running and there is at least one attacker and one enemy. Target's HP is more than 0.

Procedure	1. Start a battle with 1 hero against 1 enemy. 2. Record target HP before attack. 3. Call handleAttack(attacker).
Expected Outcome	Enemy's HP decreases, but not below 0.
Notes	

Test ID	BS-TC-02
Category	Battle System (HP Floor at Zero)
Requirements Coverage	UC3-Attack-Damages-HP
Initial Condition	Extremely strong hero (attack=999) vs very weak enemy (HP=5).
Procedure	1. Call processTurn(ATTACK).
Expected Outcome	Enemy HP equals 0 not a negative value.
Notes	Math.max(0, ...) guard is required.

Test ID	BS-TC-03
Category	Battle System (Defend Action)
Requirements Coverage	UC3-Defend-Action
Initial Condition	Hero at reduced HP (50) and reduced mana (10).
Procedure	1. Call handleDefend(hero).
Expected Outcome	HP restored to 60 (+10); mana restored to 15 (+5).
Notes	

Test ID	BS-TC-04
Category	Battle System (Wait Action)
Requirements Coverage	UC3-Wait-Action
Initial Condition	Battle running with a live hero.
Procedure	1. Call handleWait(hero).

Expected Outcome	Return message contains 'waits'.
Notes	

Test ID	BS-TC-05
Category	Battle System (Heroes Win Condition)
Requirements Coverage	UC3-Battle-End
Initial Condition	Enemy HP reduced to 0.
Procedure	1. enemy.takeDamage(999). 2. Check isBattleOver().
Expected Outcome	isBattleOver() returns true. getWinner() returns 'Heroes win!'.
Notes	

Test ID	BS-TC-06

Category	Battle System (Enemies Win Condition)
Requirements Coverage	UC3-Battle-End
Initial Condition	Hero HP reduced to 0.
Procedure	1. hero.takeDamage(999). 2. Check isBattleOver().
Expected Outcome	isBattleOver() returns true. getWinner() returns 'Enemies win!'.
Notes	

Test ID	BS-TC-07
Category	Battle System (Alive Party Detection)
Requirements Coverage	UC3-Battle-End
Initial Condition	The hero is alive (HP > 0).
Procedure	1. Call noLivingUnits(List.of(hero)).

Expected Outcome	Returns false, the party is still alive.
Notes	

Test ID	BS-TC-08
Category	Battle System (Damage Formula)
Requirements Coverage	UC3-Attack-Damages-HP
Initial Condition	Hero attack=15, enemy defense=3, enemy HP=50.
Procedure	1. Call processTurn(ATTACK). 2. Measure HP change.
Expected Outcome	Damage = max(1, attacker.attack - defender.defense) = 12.
Notes	Formula: max(1, atk - def).

Test ID	BS-TC-09

Category	Battle System (Turn Order by Level)
Requirements Coverage	UC3-Turn-Order
Initial Condition	Two heroes: Novice (level 1) and Master (level 9).
Procedure	1. Initialise battle. 2. Call getTurnOrder().
Expected Outcome	First unit in turn order is Master (highest level).
Notes	

Test ID	BS-TC-10
Category	Battle System (Stun Skips Turn)
Requirements Coverage	UC3-Status-Effects
Initial Condition	The hero is stunned.
Procedure	1. hero.setStunned(true). 2. Call processTurn(ATTACK).

Expected Outcome	The result contains 'stunned'. Stun flag cleared after skipped turn.
Notes	

Test ID	BS-TC-11
Category	Battle System (Shield Absorbs Damage)
Requirements Coverage	UC3-Status-Effects
Initial Condition	The hero has shields=20, HP=100.
Procedure	1. hero.addShield(20). 2. hero.takeDamage(15).
Expected Outcome	HP remains 100. Remaining shield = 5.
Notes	Shield depletes before HP.

Test ID	BS-TC-12

Category	Battle System (Cast Blocked No Mana)
Requirements Coverage	UC3-Cast-Action
Initial Condition	Hero mana = 0.
Procedure	1. hero.setMana(0). 2. Call processTurn(CAST).
Expected Outcome	The result contains 'insufficient mana'.
Notes	

Test ID	PVE-TC-01
Category	PvE System (Start Campaign)
Requirements Coverage	UC2-Start-Campaign
Initial Condition	The system is running and PveController exists. The hero list is available.

Procedure	1. Create PveController. 2. Call startCampaign(). 3. Call getCampaign() and then getParty().
Expected Outcome	Campaign is created. The party now exists with the selected heroes.
Notes	

Test ID	PVE-TC-02
Category	PvE System (Continue Campaign Progress)
Requirements Coverage	UC6-Continue-Campaign-Progress
Initial Condition	The database contains a saved campaign for the user.
Procedure	1. User chooses Continue Campaign in dashboard. 2. The controller calls DAO to obtain the saved campaign data.
Expected Outcome	The user's party and heroes are rebuilt and the UI brings the user to the view before they exited.
Notes	

Test ID	PVE-TC-03
Category	PvE System (Next Room Advances Counter)
Requirements Coverage	UC2-Start-Campaign
Initial Condition	Campaign is active for user.
Procedure	1. Call startCampaign(). 2. Call nextRoom(userId).
Expected Outcome	currentRoom == 1.
Notes	

Test ID	PVE-TC-04
Category	PvE System (Room Types Validation)
Requirements Coverage	UC2-Start-Campaign

Initial Condition	Campaign is active.
Procedure	1. Call startCampaign(). 2. Call nextRoom(userId).
Expected Outcome	roomType is either 'BATTLE' or 'INN' no other values.
Notes	

Test ID	PVE-TC-05
Category	PvE System (Score Calculation)
Requirements Coverage	UC2-Start-Campaign
Initial Condition	Level 1 hero, 0 gold.
Procedure	1. Call startCampaign(). 2. Call calculateScore(userId).
Expected Outcome	Score == 100. Formula: (heroLevel x 100) + (gold x 10).
Notes	

Test ID	PVE-TC-06
Category	PvE System (Hero Level Up)
Requirements Coverage	UC2-Hero-Progression
Initial Condition	Hero at level 1.
Procedure	1. Gain exactly expToNextLevel() XP.
Expected Outcome	Hero reaches level 2.
Notes	

Test ID	PVE-TC-07
Category	PvE System (Inn Room Heals)
Requirements Coverage	UC2-Inn-Rest
Initial Condition	Hero at reduced HP. Party is in an InnRoom.

Procedure	1. hero.setHp(50). 2. inn.enter(party).
Expected Outcome	HP restored to maxHp. Mana restored to maxMana.
Notes	

Test ID	PVE-TC-08
Category	PvE System (Battle Room Rewards)
Requirements Coverage	UC2-Battle-Rewards
Initial Condition	BattleRoom containing a level-3 Goblin.
Procedure	1. Calculate totalExp and totalGold.
Expected Outcome	Exp == 150 (50 x 3). Gold == 225 (75 x 3).
Notes	Formulas: Exp = 50 x level, Gold = 75 x level.

Test ID	PVE-TC-09
Category	PvE System (Party Cumulative Level)
Requirements Coverage	UC2-Hero-Progression
Initial Condition	Party with heroes at level 3 and level 5.
Procedure	1. Call party.getCumulativeLevel().
Expected Outcome	Returns 8 (3 + 5).
Notes	

Test ID	PVE-TC-10
Category	PvE System (Restore Campaign State)
Requirements Coverage	UC6-Continue-Campaign-Progress
Initial Condition	Saved campaign state exists with room=15 and gold=750.
Procedure	1. Build SavedStateRequest with room=15, gold=750.

	2. Call restoreCampaign(userId, savedState).
Expected Outcome	Response isSuccess=true. currentRoom=15, gold=750.
Notes	

Test ID	PVP-TC01
Category	PvP Invitation (Send Invite)
Requirements Coverage	PC-Send-Invite
Initial Condition	System has been initiated and runs, available database connection, both users exist.
Procedure	1. User A enters User B username. 2. User A sends the invite. 3. System checks if User B exists and invite requirements are met.
Expected Outcome	A PvP invite is created and stored. User B gets the invite notification.
Notes	Users should have valid profiles.

Test ID	PVP-TC02
Category	PvP Invitation (Accept Invite)
Requirements Coverage	PC-Accept-Invite
Initial Condition	System has been initiated and runs, available database connection, a valid PvP invite exists.
Procedure	1. User B receives PvP invite. 2. User B accepts invite. 3. System verifies if invite is valid.
Expected Outcome	PvP status is updated to accepted and both users are sent to a new battle session.
Notes	

Test ID	PVP-TC-03
Category	PvP (Send Invite PENDING Status)

Requirements Coverage	PC-Send-Invite
Initial Condition	PvP service running. Both users exist.
Procedure	1. POST /api/pvp/invite with senderId and target username.
Expected Outcome	HTTP 200. status=PENDING. Numeric inviteId returned.
Notes	

Test ID	PVP-TC-04
Category	PvP (Accept Valid Invite)
Requirements Coverage	PC-Accept-Invite
Initial Condition	A valid PvP invite exists.
Procedure	1. Create invite to obtain a real inviteId. 2. POST /api/pvp/accept with the real inviteId.
Expected Outcome	HTTP 200. status=ACCEPTED.

Notes	

Test ID	PVP-TC-05
Category	PvP (Accept Non-Existent Invite)
Requirements Coverage	PC-Accept-Invite
Initial Condition	No invite with ID 99999.
Procedure	1. POST /api/pvp/accept with inviteId=99999.
Expected Outcome	HTTP 400. JSON error='Invite not found'.
Notes	

Test ID	PVP-TC-06
Category	PvP (Record Result)

Requirements Coverage	PC-Record-Result
Initial Condition	PvP service running.
Procedure	1. POST /api/pvp/result with winnerId=1, loserId=2.
Expected Outcome	HTTP 200. status=RECORDED. winnerUserId=1. loserUserId=2.
Notes	

Test ID	PVP-TC-07
Category	PvP Invitation (Accept Non-existing Invite)
Requirements Coverage	PC-Accept-Invite
Initial Condition	The invite ID does not exist in the system
Procedure	1. POST /api/pvp/accept is called with a fake invite ID. 2. System checks if invite exists
Expected Outcome	404 Not Found
Notes	

Test ID	PVP-TC-08
Category	PvP Match Result (Record Result)
Requirements Coverage	PC-Accept-Invite
Initial Condition	A completed battle exists between two players
Procedure	1. POST /api/pvp/result is called with winner and loser usernames
Expected Outcome	Response has status "RECORDED" with winnerUsername and loserUsername matching the input
Notes	

Test ID	IT-TC-01
Category	System Integration (User Registration)
Requirements Coverage	UC1-Profile-Creation
Initial Condition	1. The system is running with all services healthy. 2. The username does not already exist.

Procedure	POST /api/profile/register with a unique username and password
Expected Outcome	HTTP 200. Response contains success = true and the registered username
Notes	

Test ID	IT-TC-02
Category	System Integration (Duplicate Registration)
Requirements Coverage	UC1-Profile-Creation
Initial Condition	The username from IT-TC-01 already exists in the database.
Procedure	POST /api/profile/register with the same username used in IT-TC-01.
Expected Outcome	HTTP 400. Response contains success = false.
Notes	

Test ID	IT-TC-03

Category	System Integration (Login and Profile Data)
Requirements Coverage	UC1-Profile-Login
Initial Condition	The user from IT-TC-01 is registered.
Procedure	POST /api/profile/login with the correct username and password.
Expected Outcome	HTTP 200. Response contains success = true, a valid userId, scores = 0, and campaignProgress = 0.
Notes	

Test ID	IT-TC-04
Category	System Integration (Start PvE Campaign)
Requirements Coverage	UC2-Start-Campaign
Initial Condition	User from IT-TC-01 is logged in. No active campaign exists.
Procedure	POST /api/pve/{userId}/start with a single WARRIOR hero named TEST at level 1.
Expected Outcome	HTTP 200. Response contains success = true and currentRoom = 0.

Notes	

Test ID	IT-TC-05
Category	System Integration (Advance to Next Room)
Requirements Coverage	UC2-Start-Campaign
Initial Condition	An active campaign exists for the user with currentRoom = 0.
Procedure	POST /api/pve/{userId}/next-room.
Expected Outcome	HTTP 200. Response contains success = true, currentRoom = 1, and roomType is either BATTLE or INN.
Notes	

Test ID	IT-TC-06
Category	System Integration (Battle Service Health)
Requirements Coverage	UC3-Battle-System

Initial Condition	The full Docker stack is running.
Procedure	GET /api/battle/health directly against the battle service on port 5001
Expected Outcome	HTTP 200.
Notes	

Test ID	IT-TC-07
Category	System Integration (Save Campaign State)
Requirements Coverage	UC5-Save-Campaign-Progress
Initial Condition	The system is running. User exists with userId from IT-TC-03
Procedure	POST /api/data/campaign/save with userId, partyName "IntTestParty", currentRoom = 1, gold = 500, and one hero
Expected Outcome	HTTP 200. Response contains partyName = "IntTestParty" and currentRoom = 1
Notes	

Test ID	IT-TC-08
Category	System Integration (Load Saved Campaign)
Requirements Coverage	UC6-Continue-Campaign-Progress
Initial Condition	Campaign was saved in IT-TC-07 with room = 1 and gold = 500
Procedure	GET /api/data/campaign/{userId}
Expected Outcome	HTTP 200. Response contains partyName = "IntTestParty", currentRoom = 1, and gold = 500
Notes	

Test ID	IT-TC-09
Category	System Integration (Save and Retrieve Best Score)
Requirements Coverage	UC2-Score-Calculation
Initial Condition	The user exists. No score previously saved

Procedure	1. POST /api/data/scores/{userId} with score = 1500. 2. GET /api/data/scores/{userId}/best
Expected Outcome	First call returns HTTP 200. Second call returns HTTP 200 with bestScore = 1500
Notes	

Test ID	IT-TC-10
Category	System Integration (Leaderboard)
Requirements Coverage	UC1-Profile-Rankings
Initial Condition	At least one score has been saved (IT-TC-09 has run)
Procedure	GET /api/data/scores/top?limit=5
Expected Outcome	HTTP 200. Response is a non-empty list
Notes	

Test ID	IT-TC-11

Category	System Integration (PvP Invite — Invalid Target)
Requirements Coverage	UC7-PvP-Invitation
Initial Condition	The system is running. Target username "INVITESUBJECT" does not exist
Procedure	POST /api/pvp/invite with fromUsername set to the test user and toUsername = "INVITESUBJECT"
Expected Outcome	HTTP 400. Response contains an error message referencing "not found"
Notes	

Test ID	IT-TC-12
Category	System Integration (Gateway Routing)
Requirements Coverage	UC1-Profile-Creation
Initial Condition	The user from IT-TC-01 exists. Gateway is running on port 8080
Procedure	GET /api/profile/{userId} through the gateway
Expected Outcome	HTTP 200. Response contains username matching the test user

Notes	

Installation Manual

Prerequisites

- **Docker Desktop (Version 24 +)**
- **Git**
- **Java 17+**
- **Ports 8080 and 3306 must not be in use**

Cloning Repository
Open terminal and run:

```
git clone https://github.com/Kevin-Bui1/CloudComputing-Legend-of-Sword-and-Wand.git
cd CloudComputing-Legend-of-Sword-and-Wand
```

Starting System
Make sure Docker Desktop is open and running before proceeding.
Run the command below to build all Docker images and start all services:

```
docker compose up --build
```

After running the command, wait about a minute for all services and the database to finish starting.

Verify System Running
Run the command below to check if services are healthy

```
docker compose ps
```

All 6 containers should show a healthy status or running.
Additionally, to test the gateway directly open a browser and go to

```
http://localhost:8080
```

This will open the web client where you can register and play the game right away.

Running Tests
To run unit tests for a service use the commands below

```
cd battle-service
mvn test
```

Replace battle-service with any other service or integration-tests folder name to test that specific service.

Stop System
To stop all containers while keeping saved data

```
docker compose down
```

To stop all containers and delete all data including the database

```
docker compose down -v
```

Troubleshooting
Port already in use
If port 8080 or 3306 is already in use on your machine, open docker-compose.yml and change the host port mapping. For example, to change the gateway to port 9090:

```
ports:
  - "9090:8080"
```

Then access the client at http://localhost:9090 instead.

Use of AI

AI Tool Used: Claude AI
Date of Use: March 23, 2026
Codebase Assistance: AI agent was asked to help fix our unit tests implementation for integration tests.

AI Tool Used: Claude AI
Date of Use: March 25, 2026
Codebase Assistance: AI agent was asked to spot errors and assist in implementing ProfileController.java as returns were not working.

AI Tool Used: Claude AI
Date of Use: April 1, 2026
Codebase Assistance: AI agent was asked to review documentation to suggest what components were missing and partially completed.